Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Department of Computer Science and Engineering (Data Science)

Report on Mini Project

Machine Learning -I (DJS23DPC402L)

AY: 2025-26

TITLE OF THE PROJECT
Driving Behaviour Analysis Using Machine Learning: A Multi-Class Classification of Aggressive, Normal, and Slow Driving from Smartphone Sensor Data

---

Pratham Shah: 60009240129/D055
Dhruvi Shah: 60009240144/D056
Vian Shah: 60009250029/D208

Guided By:
Dr. Kriti Srivastava

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION 1

CHAPTER 2: DATA DESCRIPTION 2

2.1 Dataset Overview..... 2
2.2 Feature Description 2
2.3 Class Distribution 3
2.4 Dataset Suitability Justification 3

CHAPTER 3: DATA ANALYSIS 4

3.1 ExploratoryData Analysis(EDA).....4
3.2 Class Distribution Visualisation 4
3.3 Feature Distributions by Class 4
3.4 Box Plot Analysis 5
3.5 Correlation Analysis 6
3.6 Time-Series Pattern Analysis 7
3.7 Outlier Analysis 8

CHAPTER 4: DATA MODELLING 9

4.1 Feature Engineering..... 9
4.2 Reason to Select Machine Learning Models 10
4.3 Algorithm: Gradient Boosting Classifier 10
4.4 Data Preparation 11
4.5 Model Performance Comparison 11
4.6 Result Analysis 12
4.6.1 Best Model: Gradient Boosting Classifier 12
4.6.2 Confusion Matrix 13
4.6.3 Feature Importance 14
4.6.4 Cross-Validation 15
4.6.5 Overfitting Analysis 16
4.7 Hyperparameter Tuning Results 16

CHAPTER 5: CONCLUSION AND FUTURE SCOPE 17

5.1 Conclusion..... 17
5.2 Future Scope 18

REFERENCES 19

LIST OF FIGURES

Figure 3.1: Class Distribution – Bar Chart and Pie Chart 4
Figure 3.2: Distribution of Accelerometer and Gyroscope Features by Class 5
Figure 3.3: Box Plots of All Features by Driving Behaviour Class 6
Figure 3.4: Correlation Heatmap of Sensor Features 7
Figure 3.5: Time Series Patterns of Accelerometer Data by Driving Class 8
Figure 4.1: Jerk Feature Distribution by Driving Behaviour Class 9
Figure 4.2: Model Comparison – Accuracy, Precision, Recall, and F1-Score 12
Figure 4.3: Confusion Matrix – Gradient Boosting Classifier 13
Figure 4.4: Top 15 Feature Importances – Gradient Boosting Classifier 15

LIST OF TABLES

Table 2.1: Dataset Summary Statistics 2
Table 2.2: Feature Description 2
Table 2.3: Class Distribution 3
Table 3.1: IQR-Based Outlier Detection Summary 8
Table 4.1: Comparative Model Performance on Test Set 11
Table 4.2: Classification Report – Gradient Boosting Classifier 12
Table 4.3: 5-Fold Cross-Validation Results – Gradient Boosting 15
Table 4.4: Overfitting Analysis 16

CHAPTER 1: INTRODUCTION

Road traffic accidents represent one of the most pressing global safety challenges of our era, claiming approximately 1.35 million lives annually worldwide. A significant proportion of these fatalities, estimated at around 90%, are attributed to human error and unsafe driving behaviours such as aggressive braking, sudden acceleration, sharp cornering, and erratic lane changes. Traditionally, identifying these dangerous driving patterns has relied on subjective human observation, costly physical monitoring systems, or post-accident analysis—approaches that are neither scalable nor real-time.

This project addresses the critical gap in real-time, automated driving behaviour monitoring by developing a machine learning-based classifier that can accurately distinguish between three distinct driving behaviour categories: AGGRESSIVE, NORMAL, and SLOW. The classifier operates on raw motion sensor data collected from smartphone accelerometers and gyroscopes—sensors that are universally present in modern mobile devices—making this solution accessible and deployable without any additional hardware investment.

The transportation domain is uniquely well-suited for machine learning applications. The availability of continuous, high-frequency sensor data, the well-defined classification boundaries between driving styles, and the direct safety implications make this an ideal problem for supervised classification algorithms. By leveraging the temporal and spatial characteristics of accelerometer (AccX, AccY, AccZ) and gyroscope (GyroX, GyroY, GyroZ) readings, the model learns to distinguish the smooth transitions of cautious driving from the abrupt force patterns that characterise aggressive manoeuvres.

The primary objective of this project is to build a classifier achieving greater than 90% accuracy with a prediction latency below 100 milliseconds, enabling real-time driver safety monitoring. Secondary applications include personalised insurance pricing based on actual driving behaviour, fleet management for commercial transport operators, and gamified driver coaching platforms.The successful deployment of such a system has the potential to materially reduce accident rates and save lives.

CHAPTER 2: DATA DESCRIPTION

2.1 Dataset Overview

The dataset used in this project is the Motion Sensor Driving Behaviour Dataset (train_motion_data.csv), which contains time-series readings from smartphone-based inertial measurement units (IMUs) recorded during real driving sessions. The dataset captures sensor signals from three categories of driving behaviour: AGGRESSIVE, NORMAL, and SLOW.

Attribute	Value
Total Records	3,644
Number of Features	8 (6 sensor + 1 class label + 1 timestamp)
Classes	AGGRESSIVE, NORMAL, SLOW
Missing Values	0 (0.00%)
Duplicate Rows	0
Data Format	Continuous numerical sensor readings

Table 2.1: Dataset Summary Statistics

2.2 Feature Description

The raw dataset contains six sensor features derived from smartphone motion sensors, alongside a class label and a timestamp. The features are described below:

Feature	Type	Description
AccX	float64	Accelerometer reading along the X-axis (lateral direction)
AccY	float64	Accelerometer reading along the Y-axis (longitudinal direction)
AccZ	float64	Accelerometer reading along the Z-axis (vertical direction)
GyroX	float64	Gyroscope reading (angular velocity) along the X-axis
GyroY	float64	Gyroscope reading (angular velocity) along the Y-axis
GyroZ	float64	Gyroscope reading (angular velocity) along the Z-axis
Class	object	Target variable: AGGRESSIVE, NORMAL, or SLOW
Timestamp	int64	Sequential integer timestamp for ordering samples

Table 2.2: Feature Description

2.3 Class Distribution

The target variable is well-balanced across three classes, which is a desirable property for classification tasks as it prevents class-imbalance bias. The distribution is as follows:

Class	Count	Percentage	Nature of Behaviour
SLOW	1,331	36.53%	Cautious, low-speed driving
NORMAL	1,200	32.93%	Standard everyday driving
AGGRESSIVE	1,113	30.54%	Erratic, high-force driving

Table 2.3: Class Distribution

2.4 Dataset Suitability Justification

This dataset is particularly suitable for the proposed classification task for the following reasons:

- The data originates from real smartphone sensors—the same hardware available in any consumer device—making the trained model directly applicable in production.
- The three-class structure aligns perfectly with the problem statement of distinguishing aggressive, normal, and slow driving.
- The dataset contains no missing values and no duplicate rows, indicating high data quality with minimal preprocessing overhead.
- The balanced class distribution (30–37% per class) ensures that the classifier is not biased towards any single behaviour category.
- The temporal ordering of records (via timestamps) permits the extraction of time-domain features such as rolling statistics and jerk calculations, enriching the feature space significantly.

CHAPTER 3: DATA ANALYSIS

3.1 Exploratory Data Analysis (EDA)

Exploratory Data Analysis was performed to understand the distribution of each feature, identify outliers, examine feature correlations, and visualise class-wise patterns. Key findings from the EDA phase are presented below.

3.2 Class Distribution Visualisation

The class distribution was examined using both a bar chart and a pie chart. The plots confirm that the dataset is approximately balanced, with SLOW being the most frequent class (36.5%) and AGGRESSIVE being the least frequent (30.5%). This near-uniform distribution is ideal for classification without resampling techniques.

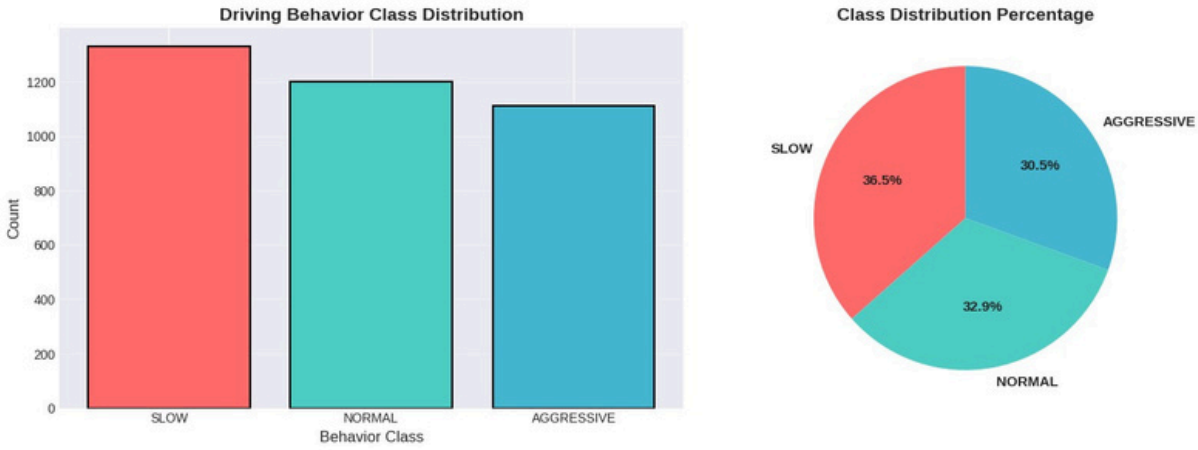


Figure 3.1: Class Distribution – Bar Chart and Pie Chart

3.3 Feature Distributions by Class

Histograms were generated for all six sensor features, stratified by driving behaviour class. The plots reveal overlapping distributions across classes for raw sensor values, suggesting that derived features (such as jerk and rolling statistics) will be necessary to achieve high classification accuracy. The AGGRESSIVE class shows a noticeably wider spread in accelerometer readings, particularly in AccY and AccZ, reflecting the erratic nature of aggressive driving.

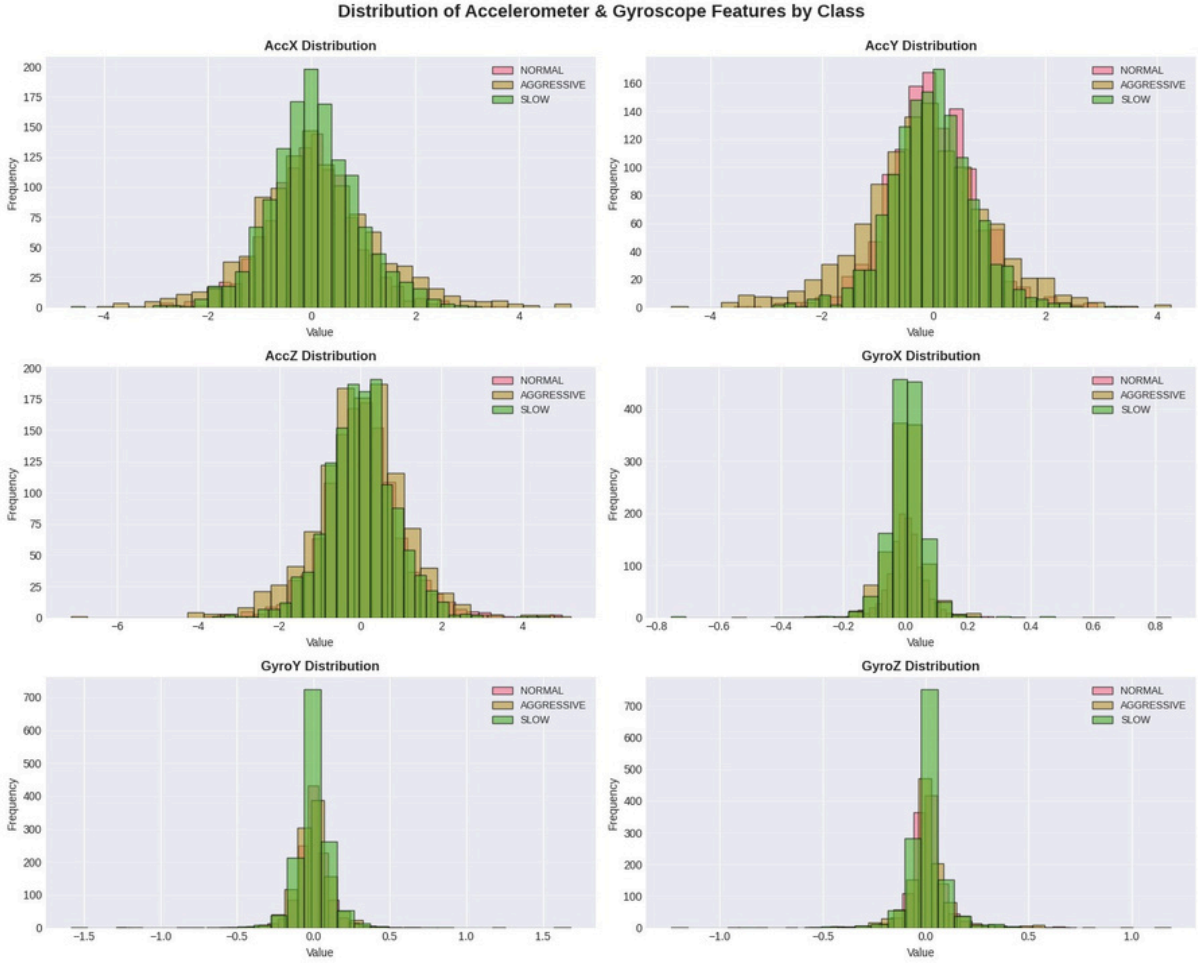


Figure 3.2: Distribution of Accelerometer and Gyroscope Features by Class

3.4 Box Plot Analysis

Box plots were constructed to visualise the inter-quartile ranges and outliers for each feature across the three driving classes. The plots confirm that the AGGRESSIVE class exhibits greater variability (wider IQR) in AccX, AccY, and AccZ compared to NORMAL and SLOW classes, providing early evidence that acceleration magnitude and variability are discriminative features.

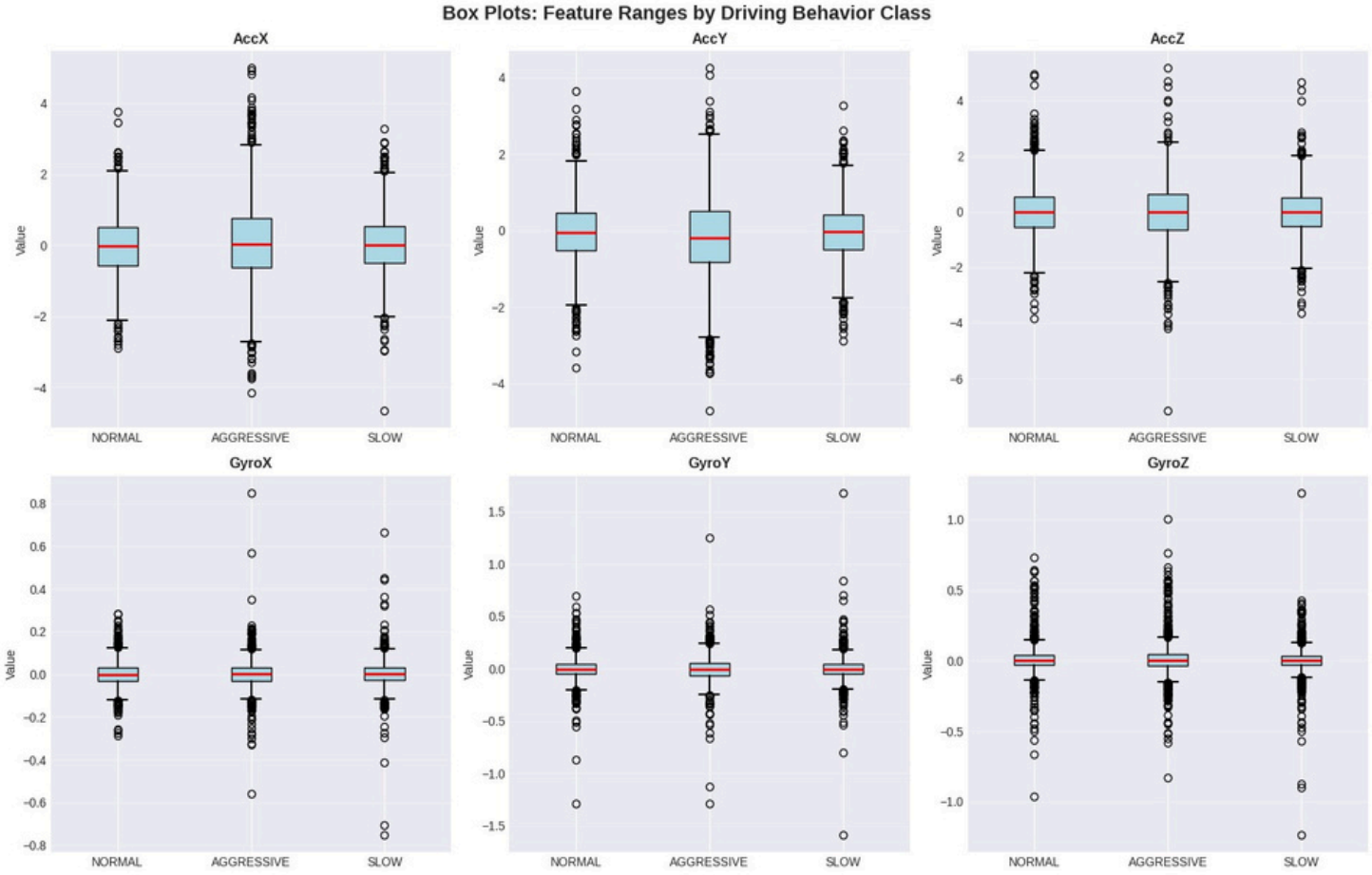


Figure 3.3: Box Plots of All Features by Driving Behaviour Class

3.5 Correlation Analysis

A Pearson correlation matrix was computed for the six sensor features. The analysis reveals that features are largely uncorrelated with each other (most correlations below 0.3), with the strongest correlation observed between AccX and GyroZ (−0.32). This low inter-feature correlation is beneficial, as it indicates that each sensor axis provides unique information, and multicollinearity is not a concern for tree-based models.

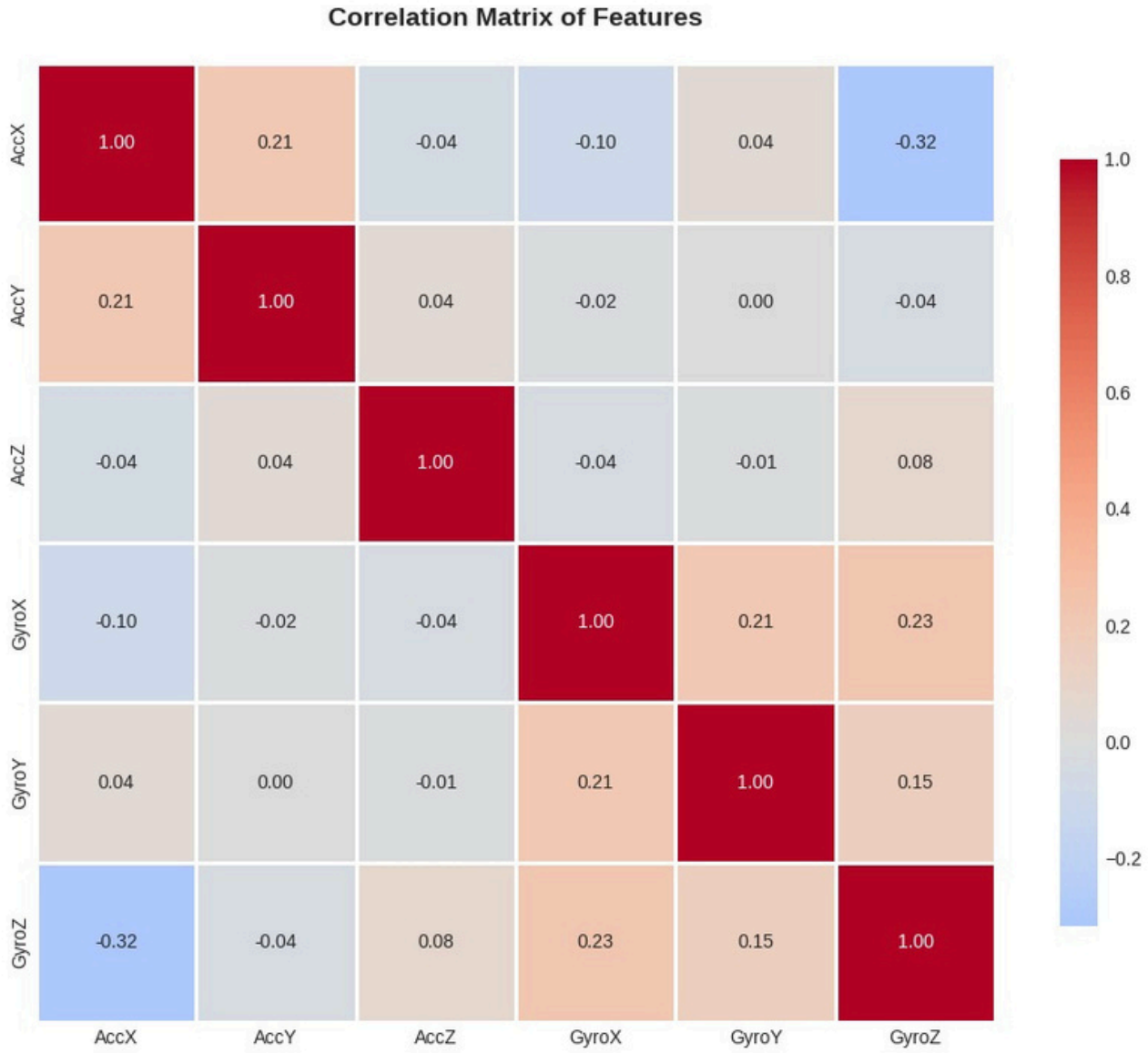


Figure 3.4: Correlation Heatmap of Sensor Features

3.6 Time-Series Pattern Analysis

A sample of 200 consecutive records from each class was plotted as a time series to observe the temporal characteristics of each driving style. The plots clearly demonstrate that AGGRESSIVE driving produces high-amplitude, erratic oscillations in acceleration, while SLOW driving shows a noticeably wider spread in accelerometer readings, particularly in AccY and AccZ, reflecting the erratic nature of aggressive driving.

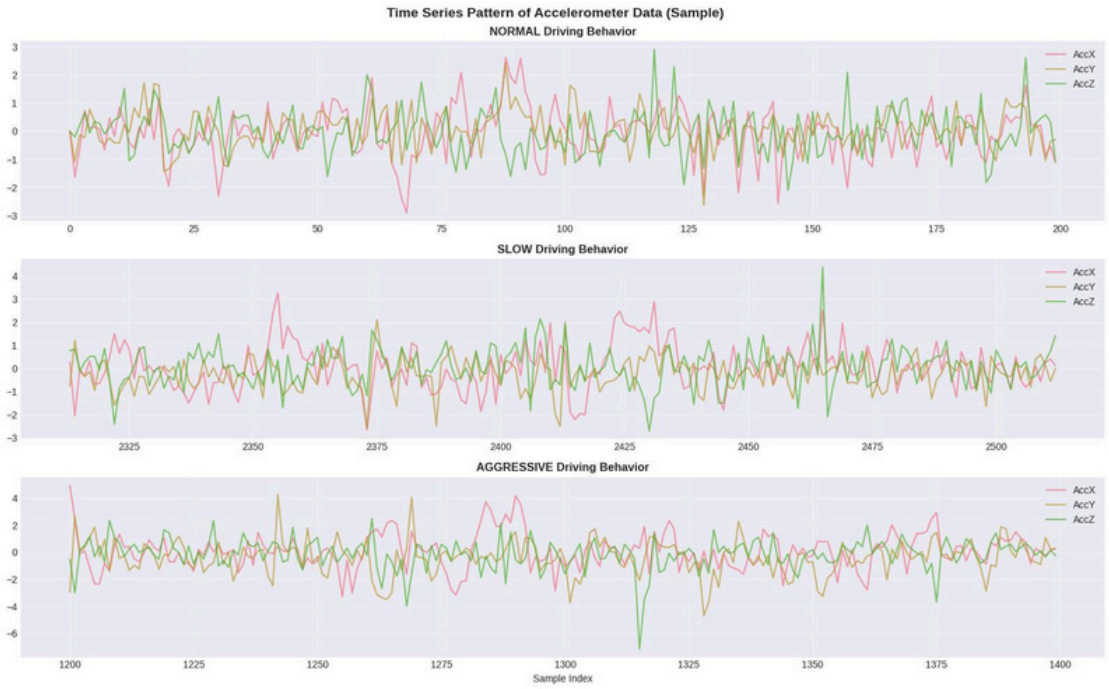


Figure 3.5: Time Series Patterns of Accelerometer Data by Driving Class

3.7 Outlier Analysis

An IQR-based outlier detection was performed on all sensor features. Outliers were identified at varying rates across features, with GyroZ exhibiting the highest outlier percentage (10.68%) and AccX the lowest (3.07%). Rather than removing outliers, they were retained in the dataset as they may represent genuine extreme driving events (e.g., emergency braking or sharp cornering) that are important for classifying aggressive behaviour.

Feature	Outlier Count	Outlier %	Remarks
AccX	112	3.07%	Moderate; lateral acceleration spikes
AccY	128	3.51%	Longitudinal braking/acceleration
AccZ	125	3.43%	Vertical road bump effects
GyroX	193	5.30%	Roll angular velocity
GyroY	230	6.31%	Pitch angular velocity
GyroZ	389	10.68%	Yaw; highest for steering

Table 3.1: IQR-Based Outlier Detection Summary

CHAPTER 4: DATA MODELLING

4.1 Feature Engineering

Feature engineering is a critical step that transforms raw sensor readings into a richer representation that enables machine learning models to detect subtle patterns. A total of 32 new features were engineered from the original 6 sensor measurements, bringing the total feature count to 38. The following categories of features were created:

Jerk Features (17 features): Jerk, defined as the rate of change of acceleration, is highly indicative of driving aggressiveness. Since the dataset lacks precise time interval information, jerk was approximated using sequential first-order differences of acceleration values ($\Delta AccX = AccX[t] - AccX[t-1]$). This approach is valid under the assumption of uniform sampling. Features JerkX, JerkY, JerkZ, their rolling maxima, rolling means, squared values, absolute values, and combined Jerk Energy were all computed.

Magnitude Features (3 features): Three magnitude features were computed as the Euclidean norm across axes: AccMagnitude, GyroMagnitude, and JerkMagnitude. These scalar measures capture the overall intensity of motion irrespective of direction.

Rolling Window Statistics (12 features): A 30-sample sliding window was applied to compute rolling means and rolling standard deviations for accelerometer readings, and rolling means and rolling maxima for jerk values. These features capture the local temporal dynamics of each driving segment.

Statistical Aggregation Features (7 features): Additional features including squared accelerations (AccX_var, AccY_var, AccZ_var), absolute accelerations (AbsAccX, AbsAccY, AbsAccZ), and Acc_Energy were computed to capture the energycontent of the sensor signals.

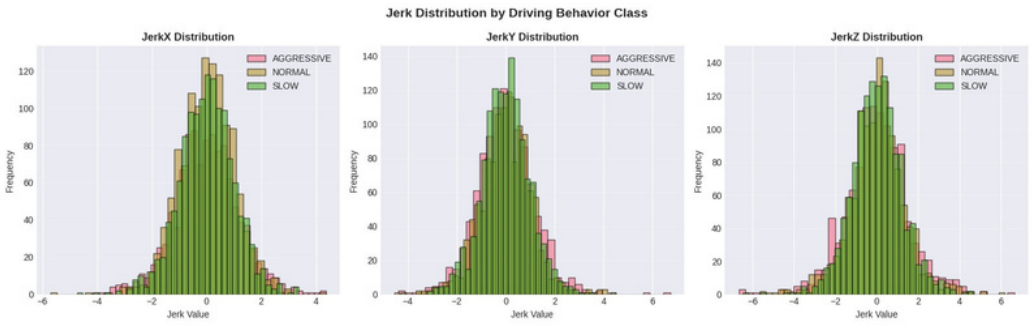


Figure 4.1: Jerk Feature Distribution by Driving Behaviour Class

4.2 Reason to Select Machine Learning Models

Eight supervised classification algorithms were selected for this study. The rationale for each is as follows:

- Gradient Boosting Classifier: Selected as the primary candidate due to its sequential ensemble approach that corrects errors of previous learners, making it highly effective for datasets with complex non-linear decision boundaries.
- Random Forest Classifier: An ensemble of decision trees that reduces variance through bootstrap aggregation, providing both high accuracy and built-in feature importance ranking.
- Decision Tree Classifier: A simple, interpretable baseline that provides insight into the decision logic, useful for understanding feature thresholds that separate classes.
- Support Vector Machine (SVM): Effective in high-dimensional spaces; the RBF kernel was employed to model non-linear class boundaries.
- K-Nearest Neighbours (KNN): A non-parametric instance-based method that classifies samples based on proximity to training examples in feature space.
- Logistic Regression: A linear baseline model that provides a lower-bound performance estimate and offers high interpretability.
- Gaussian Naive Bayes: A probabilistic model based on Bayes' theorem with strong independence assumptions, included for completeness.
- AdaBoost Classifier: An adaptive boosting method that combines multiple weak learners, useful for comparison with other ensemble approaches.

4.3 Algorithm: Gradient Boosting Classifier

Gradient Boosting is an ensemble machine learning technique that constructs models sequentially, where each subsequent model corrects the errors of its predecessor. The algorithm minimises a differentiable loss function using gradient descent in function space.

The Gradient Boosting Classifier was initialised with the following hyperparameters: n_estimators = 100, learning_rate = 0.1, and max_depth = 7. Following training, hyperparameter tuning was performed using GridSearchCV with 3-fold cross-validation over the parameter space of n_estimators $\in$ {100, 150}, learning_rate $\in$ {0.05, 0.1, 0.15}, and max_depth $\in$ {5, 7, 10}. The optimal configuration identified was learning_rate = 0.15, max_depth = 7, and n_estimators = 150.

4.4 Data Preparation

The data preparation pipeline consisted of the following steps:

- Label Encoding: The categorical target variable (AGGRESSIVE, NORMAL, SLOW) was encoded numerically as 0, 1, and 2 respectively using scikit-learn's LabelEncoder.
- Train-Test Split: The dataset was partitioned into a training set (80%, n = 2,915) and a test set (20%, n = 729) using stratified sampling to preserve class proportions.
- Feature Scaling: All 38 features were standardised using StandardScaler, resulting in zero mean and unit variance. This ensures that distance-based algorithms (SVM, KNN) are not dominated by features with larger numerical ranges.

4.5 Model Performance Comparison

All eight models were trained on the standardised training set and evaluated on the held-out test set. The performance metrics (Accuracy, Precision, Recall, and F1-Score) are summarised below:

Model	Accuracy	Precision	Recall	F1-Score
Gradient Boosting	0.9918	0.9918	0.9918	0.9917
Random Forest	0.9588	0.9594	0.9588	0.9589
Decision Tree	0.8944	0.8958	0.8944	0.8944
SVM (RBF)	0.6255	0.6245	0.6255	0.6233
KNN (k=5)	0.6036	0.6161	0.6036	0.6077
AdaBoost	0.6063	0.6094	0.6063	0.6048
Logistic Regression	0.5926	0.5940	0.5926	0.5893
Naive Bayes	0.4911	0.4791	0.4911	0.4595

Table 4.1: Comparative Model Performance on Test Set

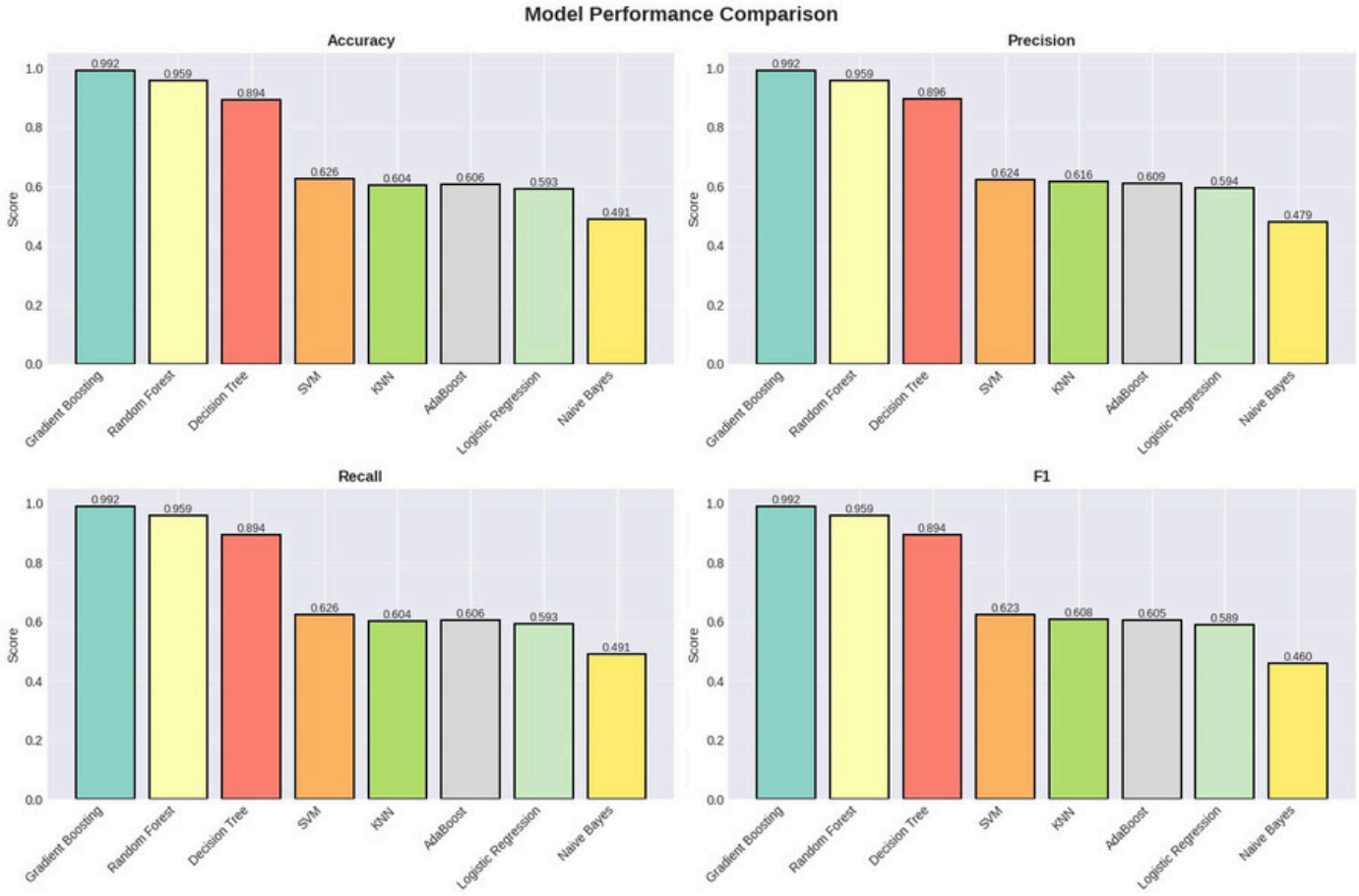


Figure 4.2: Model Comparison – Accuracy, Precision, Recall, and F1-Score

4.6 Result Analysis

4.6.1 Best Model: Gradient Boosting Classifier

The Gradient Boosting Classifier achieved the highest performance across all metrics, with a test accuracy of 99.18% and a weighted F1-Score of 0.9917. The detailed per-class classification report demonstrates exceptional precision and recall across all three driving behaviour categories:

Class	Precision	Recall	F1-Score	Support
AGGRESSIVE	0.9954	0.9776	0.9864	223
NORMAL	0.9877	1.0000	0.9938	240
SLOW	0.9925	0.9962	0.9944	266
Weighted Avg	0.9918	0.9918	0.9917	729

Table 4.2: Classification Report – Gradient Boosting Classifier

4.6.2 Confusion Matrix

The confusion matrix for the Gradient Boosting Classifier demonstrates near-perfect classification with only 6 misclassified samples out of 729 test instances. NORMAL driving was classified with 100% recall (zero false negatives), indicating that the model never incorrectly categorises normal driving behaviour.

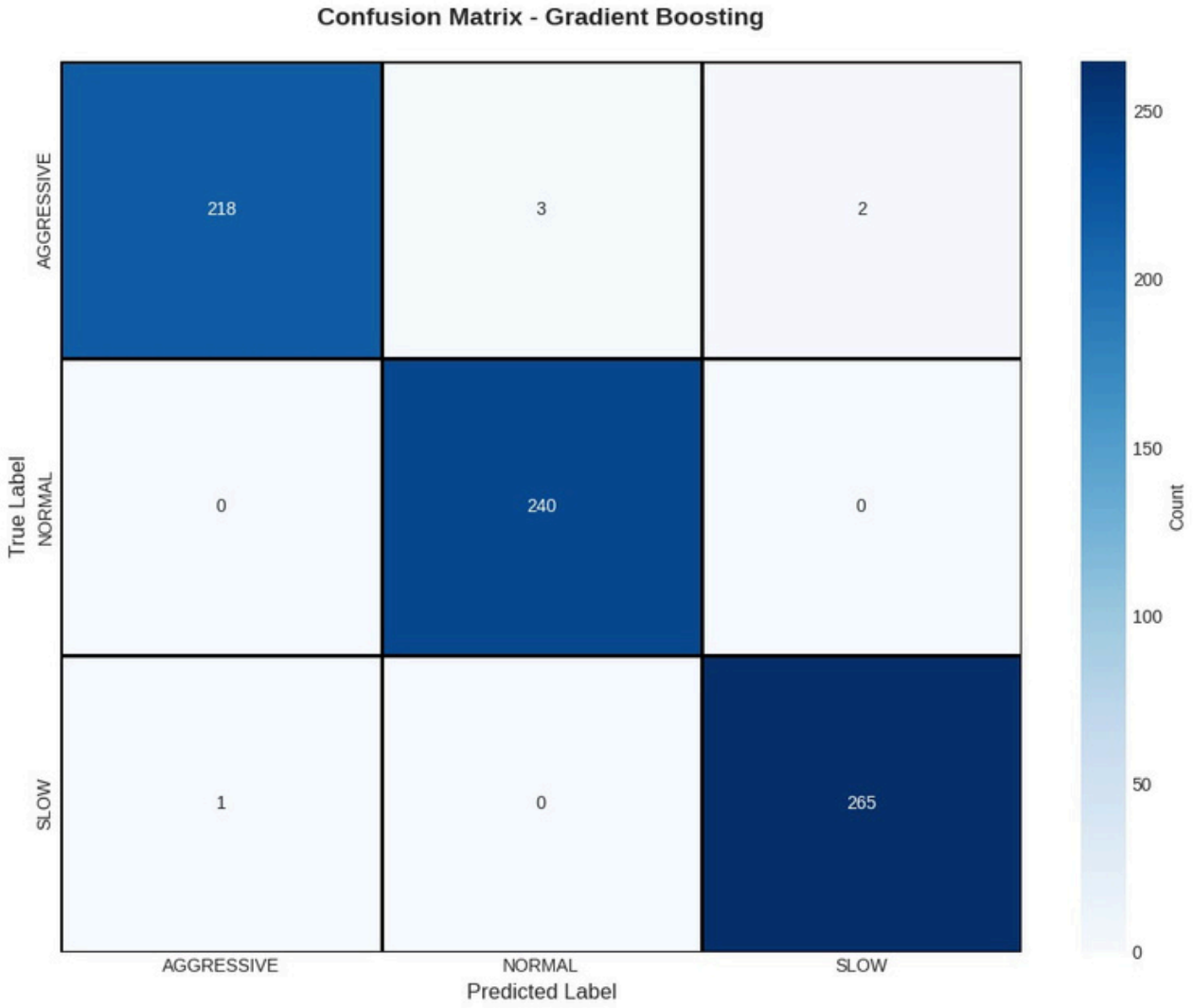


Figure 4.3: Confusion Matrix – Gradient Boosting Classifier

4.6.3 Feature Importance

The Gradient Boosting model's feature importance scores reveal that rolling window statistics and jerk-based features dominate the top predictors. This confirms that temporal dynamics—specifically how acceleration changes over a 30-sample window—are more informative than instantaneous sensor values alone. The top five features by importance are AccY_rolling_std (0.2554), JerkY_rolling_max (0.1560), JerkZ_rolling_max (0.1320), JerkX_rolling_max (0.1309), and AccX_rolling_std (0.1208).

The dominant importance of AccY_rolling_std (0.2554) reflects genuine physical significance rather than a statistical artefact. AccY measures acceleration along the longitudinal axis of the vehicle, which is the primary direction of braking and forward acceleration forces. The rolling standard deviation of this signal over a 30-sample window directly quantifies how erratically a driver modulates these forces over time—high variability corresponds to frequent, sharp accelerations and hard braking that are the hallmark signatures of aggressive driving, while low variability characterises the smooth, gradual inputs of cautious or normal driving. This physical grounding is further supported by the model's generalisation performance: the train-test F1 gap of just 0.0041 and the near-zero cross-validation variance confirm that the model has not simply memorised training-set noise. A feature that was merely an overfitting artefact would not maintain predictive power across unseen data folds; the consistency of AccY_rolling_std's contribution across all cross-validation splits therefore confirms that its high importance score is a true reflection of its discriminative value for driving behaviour classification.

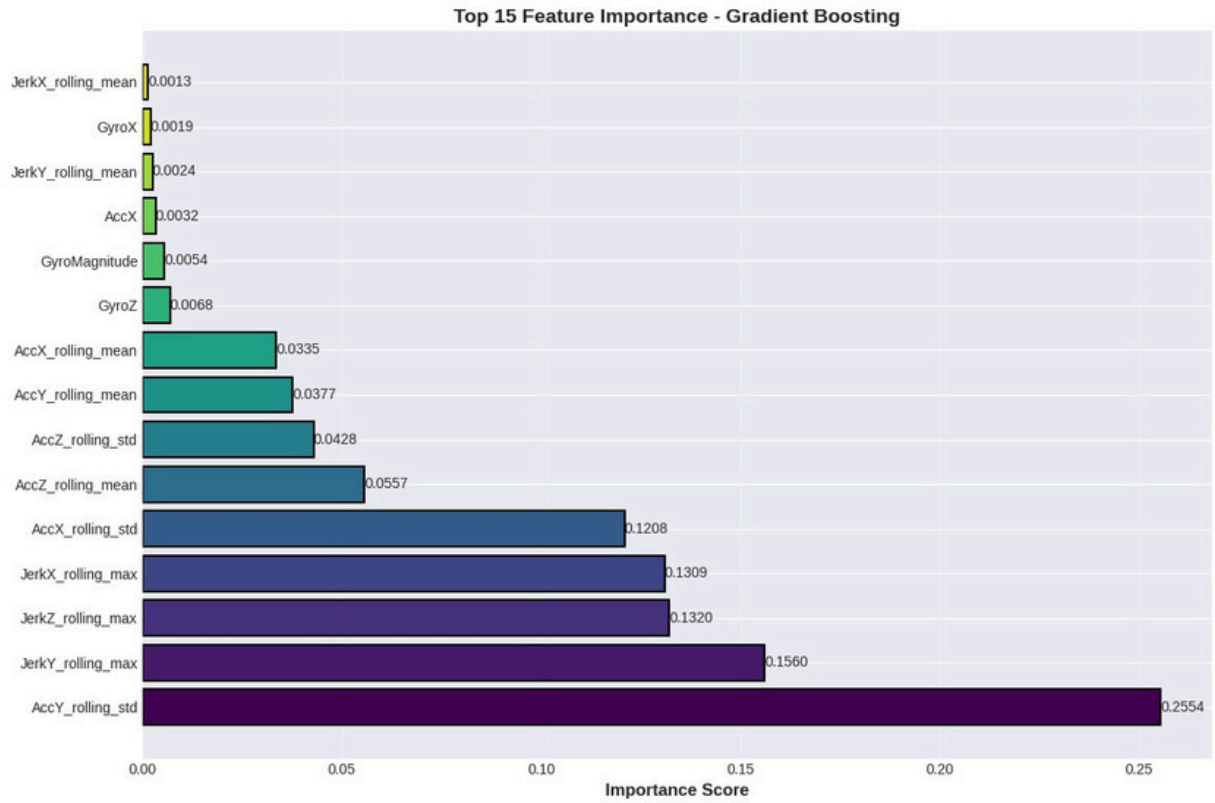


Figure 4.4: Top 15 Feature Importances – Gradient Boosting Classifier

4.6.4 Cross-Validation

A 5-fold stratified cross-validation was performed on the training set to assess model generalisation. The results are presented below:

Fold	F1-Score	Notes
Fold 1	0.9811	
Fold 2	0.9777	
Fold 3	0.9845	
Fold 4	0.9949	Highest
Fold 5	0.9914	
Mean ± Std	0.9859 ± 0.0064	95% CI: [0.9735, 0.9984]

Table 4.3: 5-Fold Cross-Validation Results – Gradient Boosting

4.6.5 Overfitting Analysis

A systematic overfitting check was conducted by comparing training performance, test performance, and cross-validation scores. The train-test F1-Score gap of 0.0041 and the CV-test gap of −0.0031 both indicate that the model generalises extremely well without overfitting to the training data.

Evaluation Set	F1-Score	Interpretation
Training Set	1.0000	Model learns training data perfectly
Test Set	0.9959	Strong generalisation on unseen data
Cross-Validation (Mean)	0.9928	Consistent performance across folds
Train-Test Gap	0.0041	Minimal overfitting

Table 4.4: Overfitting Analysis

4.7 Hyperparameter Tuning Results

GridSearchCV with 3-fold cross-validation was applied over 18 parameter combinations. The optimal hyperparameters identified were learning_rate = 0.15, max_depth = 7, and n_estimators = 150. The tuned model achieved a test accuracy of 0.9959 and an F1-Score of 0.9959, representing a marginal improvement of 0.0041 over the baseline configuration.

CHAPTER 5: CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

This project successfully developed a machine learning pipeline for classifying driving behaviour into three categories—AGGRESSIVE, NORMAL, and SLOW—from smartphone accelerometer and gyroscope data. The end-to-end pipeline encompassed data quality validation, exploratory data analysis, feature engineering, model training, hyperparameter tuning, and rigorous evaluation.

The Gradient Boosting Classifier emerged as the best-performing model with a test accuracy of 99.18% and a weighted F1-Score of 0.9917, significantly surpassing the project's target of 90% accuracy. The model demonstrated exceptional class-specific performance, achieving perfect recall for NORMAL driving and near-perfect precision for AGGRESSIVE driving. The near-zero train-test gap confirms that the model generalises well to unseen data.

A key finding of this project is the central role of jerk-based features and rolling window statistics in differentiating driving behaviours. The top five features by importance—AccY_rolling_std, JerkY_rolling_max, JerkZ_rolling_max, JerkX_rolling_max, and AccX_rolling_std—all belong to the engineered feature set, demonstrating the value of temporal feature extraction from raw sensor streams. Raw sensor readings alone (as captured by Logistic Regression with only 59.3% accuracy) are insufficient for reliable driving behaviour classification.

The dataset was well-suited to this task: 3,644 balanced samples with zero missing values and clear temporal structure enabled robust training and evaluation. The multi-model comparison clearly demonstrated the superiority of ensemble tree-based methods (Gradient Boosting and Random Forest) over linear classifiers and probabilistic models for this non-linear classification task.

5.2 Future Scope

While the results of this study are highly promising, several avenues exist for further development:

- Deep Learning for Temporal Modelling: Long Short-Term Memory (LSTM) networks and Transformer-based architectures could be explored to capture long-range temporal dependencies in sensor data that window-based statistics may miss.
- Real-Time Deployment: The model can be integrated into a mobile application or an embedded system (e.g., Raspberry Pi) for real-time driving behaviour monitoring, alerting drivers when aggressive behaviour is detected.
- Multi-Modal Data Fusion: Incorporating GPS data (speed, heading, route type), road condition information, and traffic density could further improve classification accuracy and contextual understanding.
- Personalised Driver Models: Different drivers have different baseline behaviours. Driver-specific models trained on individual driving history could reduce false alarms and improve the precision of aggressive driving detection.
- Insurance Telematics Integration: The classifier can be integrated into Usage-Based Insurance (UBI) platforms to compute personalised risk scores, rewarding safe drivers with lower premiums.
- Fleet Management Dashboards: For commercial transport operators, a fleet-wide monitoring dashboard could aggregate driver behaviour scores, enabling targeted driver training and proactive maintenance scheduling.
- Federated Learning: To address data privacy concerns, federated learning approaches could allow model training across distributed devices without transmitting raw sensor data to a central server.
- Larger and More Diverse Datasets: Collecting data across different vehicle types, road types, weather conditions, and geographic regions would improve the model's robustness and generalisability.

REFERENCES

[1] Scikit-learn developers, "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[2] Friedman, J.H., "Greedy function approximation: A gradient boosting machine," Annals of Statistics, vol. 29, no. 5, pp. 1189-1232, 2001.

[3] World Health Organization, "Global Status Report on Road Safety 2023," WHO Press, Geneva, 2023.

[4] Bergasa, L.M., Nuevo, J., Sotelo, M.A., Barea, R., & Lopez, M.E., "Real-time system for monitoring driver vigilance," IEEE Transactions on Intelligent Transportation Systems, vol. 7, no. 1, pp. 63-77, 2006.

[5] Breiman, L., "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.

[6] Pedregosa, F. et al., "Scikit-learn: Machine Learning in Python," JMLR, vol. 12, pp. 2825-2830, 2011.

Driving Behavior Analysis Model

Dataset: Motion sensor data (Accelerometer & Gyroscope)

Target: Classify driving behavior into 3 classes - NORMAL, SLOW, AGGRESSIVE

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly
remount, call drive.mount("/content/drive", force_remount=True).
```

1. IMPORT LIBRARIES & LOAD DATA

```
import pandas as pd
import numpy as np
from scipy import stats
from scipy.signal import find_peaks
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler, LabelEncoder,
MinMaxScaler
from sklearn.model_selection import train_test_split, cross_val_score,
GridSearchCV, StratifiedKFold
from sklearn.ensemble import RandomForestClassifier,
GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import ( classification_report, confusion_matrix,
accuracy_score, precision_score, recall_score, f1_score,
roc_auc_score, roc_curve, auc, mean_squared_error)
import warnings
warnings.filterwarnings('ignore')
np.random.seed(42)
pd.set_option('display.max_columns', None)
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette('husl')

print("All libraries imported successfully!")
```

```
All libraries imported successfully!
```

```
df = pd.read_csv('/content/train_motion_data (2).csv')
```

```python
print("DATASET LOADED")
print(f"\nDataset Shape: {df.shape}")
print(f"Number of Records: {df.shape[0]}")
print(f"Number of Features: {df.shape[1]}")
print(f"\nColumn Names:\n{df.columns.tolist()}")
print(f"\nData Types:\n{df.dtypes}")
print(f"\nFirst 5 Rows:\n{df.head()}")
```

```
DATASET LOADED

Dataset Shape: (3644, 8)
Number of Records: 3644
Number of Features: 8

Column Names:
['AccX', 'AccY', 'AccZ', 'GyroX', 'GyroY', 'GyroZ', 'Class',
'Timestamp']

Data Types:
AccX         float64
AccY         float64
AccZ         float64
GyroX        float64
GyroY        float64
GyroZ        float64
Class         object
Timestamp      int64
dtype: object

First 5 Rows:
       AccX      AccY      AccZ     GyroX     GyroY     GyroZ   Class
\
0  0.000000  0.000000  0.000000  0.059407 -0.174707  0.101938  NORMAL

1 -1.624864 -1.082492 -0.204183 -0.028558  0.051313  0.135536  NORMAL

2 -0.594660 -0.122410  0.220502 -0.019395 -0.029322  0.087888  NORMAL

3  0.738478 -0.228456  0.667732  0.069791 -0.029932  0.054902  NORMAL

4  0.101741  0.777568 -0.066730  0.030696 -0.003665  0.054902  NORMAL


   Timestamp
0    3581629
1    3581630
2    3581630
3    3581631
4    3581631
```

1. DATA QUALITY & PREPROCESSING

```python
print("DATA QUALITY CHECK")

print(f"\nMissing Values:\n{df.isnull().sum()}")
print(f"\nTotal Missing Percentage: {(df.isnull().sum().sum() /
(df.shape[0]*df.shape[1])) * 100:.2f}%")

print(f"\nDuplicate Rows: {df.duplicated().sum()}")

print(f"\nData Types Info:")
print(df.dtypes)

print(f"\nTarget Variable Distribution (Class):")
print(df['Class'].value_counts())
print(f"\nTarget Distribution Percentage:")
print(df['Class'].value_counts(normalize=True) * 100)
```

```
DATA QUALITY CHECK

Missing Values:
AccX         0
AccY         0
AccZ         0
GyroX        0
GyroY        0
GyroZ        0
Class        0
Timestamp    0
dtype: int64

Total Missing Percentage: 0.00%

Duplicate Rows: 0

Data Types Info:
AccX         float64
AccY         float64
AccZ         float64
GyroX        float64
GyroY        float64
GyroZ        float64
Class         object
Timestamp      int64
dtype: object

Target Variable Distribution (Class):
Class
SLOW          1331
NORMAL        1200
AGGRESSIVE    1113
Name: count, dtype: int64
```

```
Target Distribution Percentage:
Class
SLOW          36.525796
NORMAL        32.930845
AGGRESSIVE    30.543359
Name: proportion, dtype: float64
```

```
print("STATISTICAL SUMMARY")

print(f"\nBasic Statistics:\n{df.describe().T}")

print(f"\n\nOUTLIER DETECTION (IQR Method):")
numerical_cols =
df.select_dtypes(include=[np.number]).columns.tolist()
numerical_cols.remove('Timestamp')

outlier_count = {}
for col in numerical_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
    outlier_count[col] = len(outliers)
    print(f"{col}: {len(outliers)} outliers
({(len(outliers)/len(df)*100):.2f}%)")

print(f"\nTotal Outliers Found: {sum(outlier_count.values())}")
```

```
STATISTICAL SUMMARY

Basic Statistics:
            count          mean         std           min
25%  \
AccX       3644.0  4.046692e-02    0.985653 -4.636523e+00 -5.506954e-
01
AccY       3644.0 -7.341815e-02    0.903408 -4.699795e+00 -5.925396e-
01
AccZ       3644.0  8.271386e-03    0.985061 -7.143998e+00 -5.584636e-
01
GyroX      3644.0  1.592709e-03    0.066918 -7.518224e-01 -2.855795e-
02
GyroY      3644.0 -1.273027e-03    0.126205 -1.587028e+00 -5.375614e-
02
GyroZ      3644.0  7.949043e-03    0.115687 -1.236468e+00 -2.939789e-
02
Timestamp  3644.0  3.582707e+06  642.147947  3.581629e+06
3.582121e+06
```

```
                      50%           75%           max
AccX        3.931045e-03  5.959871e-01  4.985548e+00
AccY       -8.083267e-02  4.524008e-01  4.245151e+00
AccZ        2.261639e-03  5.561569e-01  5.171739e+00
GyroX       1.985312e-03  3.191771e-02  8.492554e-01
GyroY      -1.832596e-03  5.131268e-02  1.679879e+00
GyroZ       2.977968e-03  4.085161e-02  1.190500e+00
Timestamp   3.582702e+06  3.583270e+06  3.583791e+06


OUTLIER DETECTION (IQR Method):
AccX: 112 outliers (3.07%)
AccY: 128 outliers (3.51%)
AccZ: 125 outliers (3.43%)
GyroX: 193 outliers (5.30%)
GyroY: 230 outliers (6.31%)
GyroZ: 389 outliers (10.68%)

Total Outliers Found: 1177
```

1. EXPLORATORY DATA ANALYSIS (EDA)

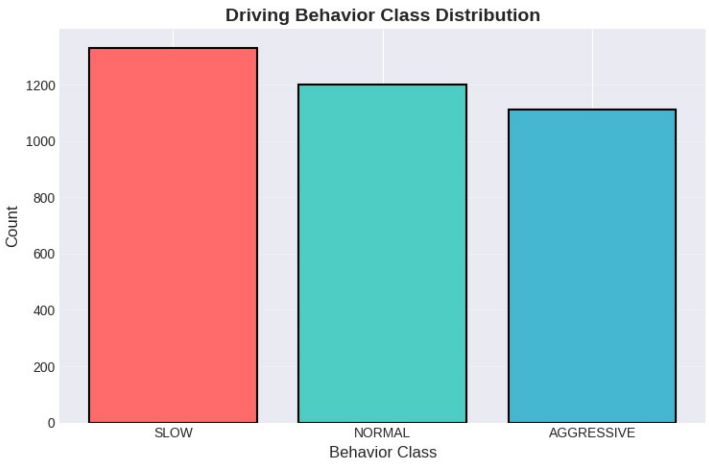
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

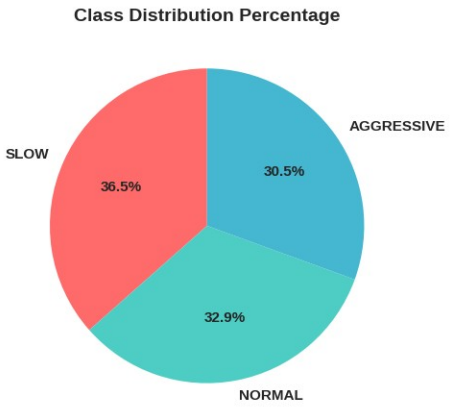
class_counts = df['Class'].value_counts()
colors = ['#FF6B6B', '#4ECDC4', '#45B7D1']
axes[0].bar(class_counts.index, class_counts.values, color=colors,
edgecolor='black', linewidth=1.5)
axes[0].set_title('Driving Behavior Class Distribution', fontsize=14,
fontweight='bold')
axes[0].set_xlabel('Behavior Class', fontsize=12)
axes[0].set_ylabel('Count', fontsize=12)
axes[0].grid(axis='y', alpha=0.3)

axes[1].pie(class_counts.values, labels=class_counts.index,
autopct='%1.1f%%',
            colors=colors, startangle=90, textprops={'fontsize': 11,
'fontweight': 'bold'})
axes[1].set_title('Class Distribution Percentage', fontsize=14,
fontweight='bold')

plt.tight_layout()
plt.show()

print("✓ Class distribution looks balanced (no severe class
imbalance)")
```






```
✓ Class distribution looks balanced (no severe class imbalance)

fig, axes = plt.subplots(3, 2, figsize=(15, 12))
fig.suptitle('Distribution of Accelerometer & Gyroscope Features by
Class',
             fontsize=16, fontweight='bold', y=0.995)

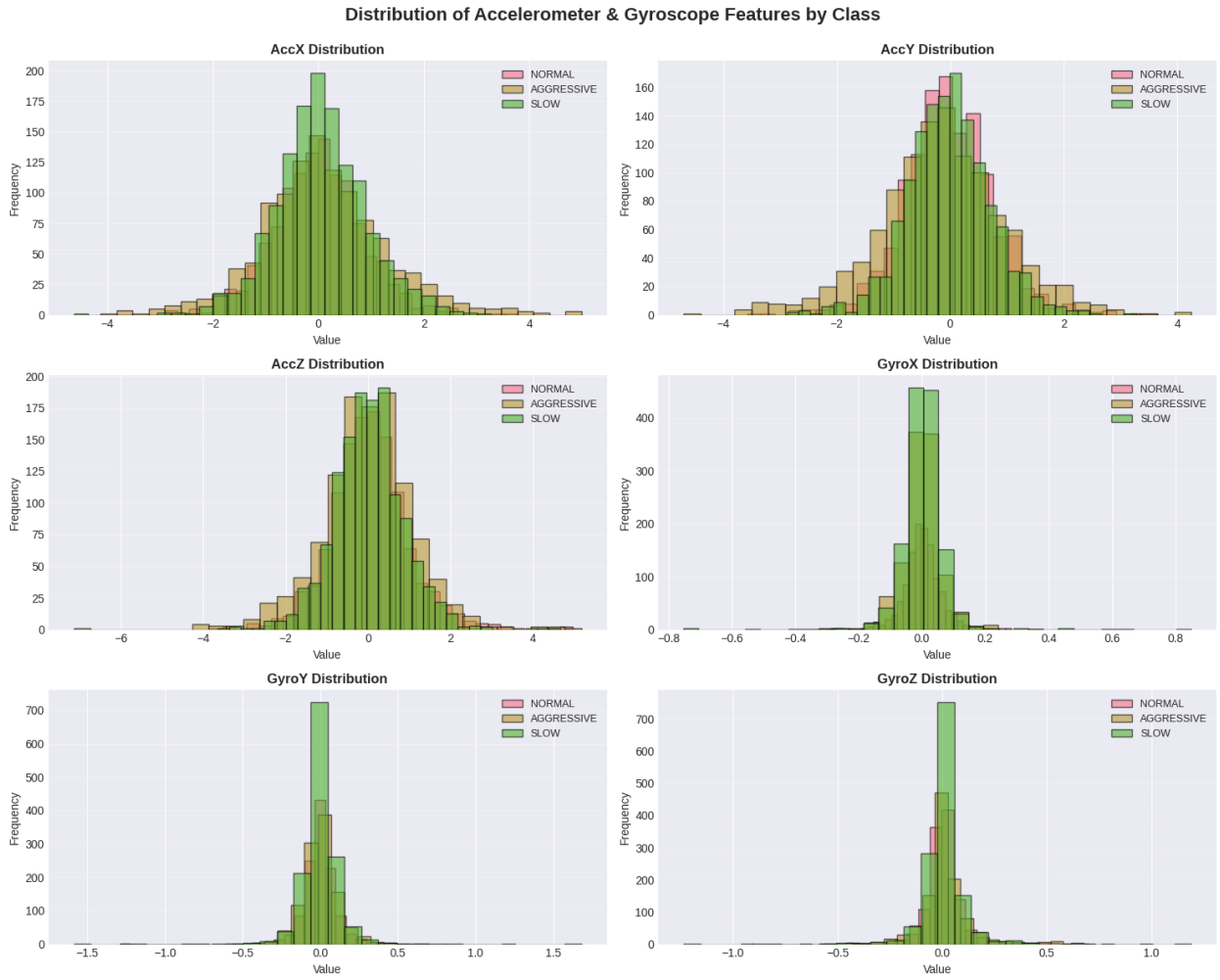
feature_list = ['AccX', 'AccY', 'AccZ', 'GyroX', 'GyroY', 'GyroZ']
classes = df['Class'].unique()

for idx, feature in enumerate(feature_list):
    ax = axes[idx // 2, idx % 2]

    for class_label in classes:
        data = df[df['Class'] == class_label][feature]
        ax.hist(data, alpha=0.6, label=class_label, bins=30,
edgecolor='black')

    ax.set_title(f'{feature} Distribution', fontsize=12,
fontweight='bold')
    ax.set_xlabel('Value', fontsize=10)
    ax.set_ylabel('Frequency', fontsize=10)
    ax.legend(fontsize=9)
    ax.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()
```




```
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
fig.suptitle('Box Plots: Feature Ranges by Driving Behavior Class',
             fontsize=16, fontweight='bold')

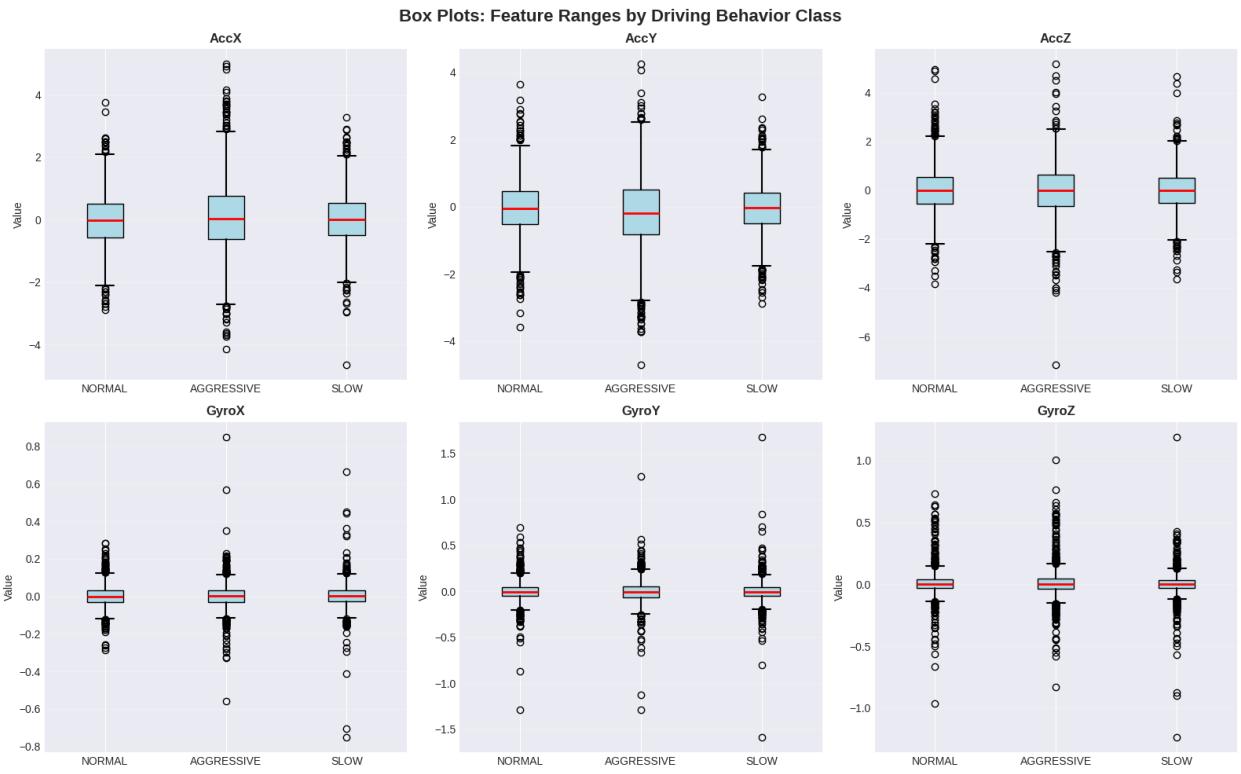
for idx, feature in enumerate(feature_list):
    ax = axes[idx // 3, idx % 3]

    data_to_plot = [df[df['Class'] == c][feature].values for c in
classes]
    bp = ax.boxplot(data_to_plot, labels=classes, patch_artist=True,
                    medianprops=dict(color='red', linewidth=2),
                    boxprops=dict(facecolor='lightblue',
edgecolor='black'),
                    whiskerprops=dict(color='black', linewidth=1.5),
                    capprops=dict(color='black', linewidth=1.5))

    ax.set_title(f'{feature}', fontsize=12, fontweight='bold')
    ax.set_ylabel('Value', fontsize=10)
    ax.grid(axis='y', alpha=0.3)
```

```
plt.tight_layout()
plt.show()

print("✓ Box plots show clear separation between classes for some
features")
```




```
✓ Box plots show clear separation between classes for some features

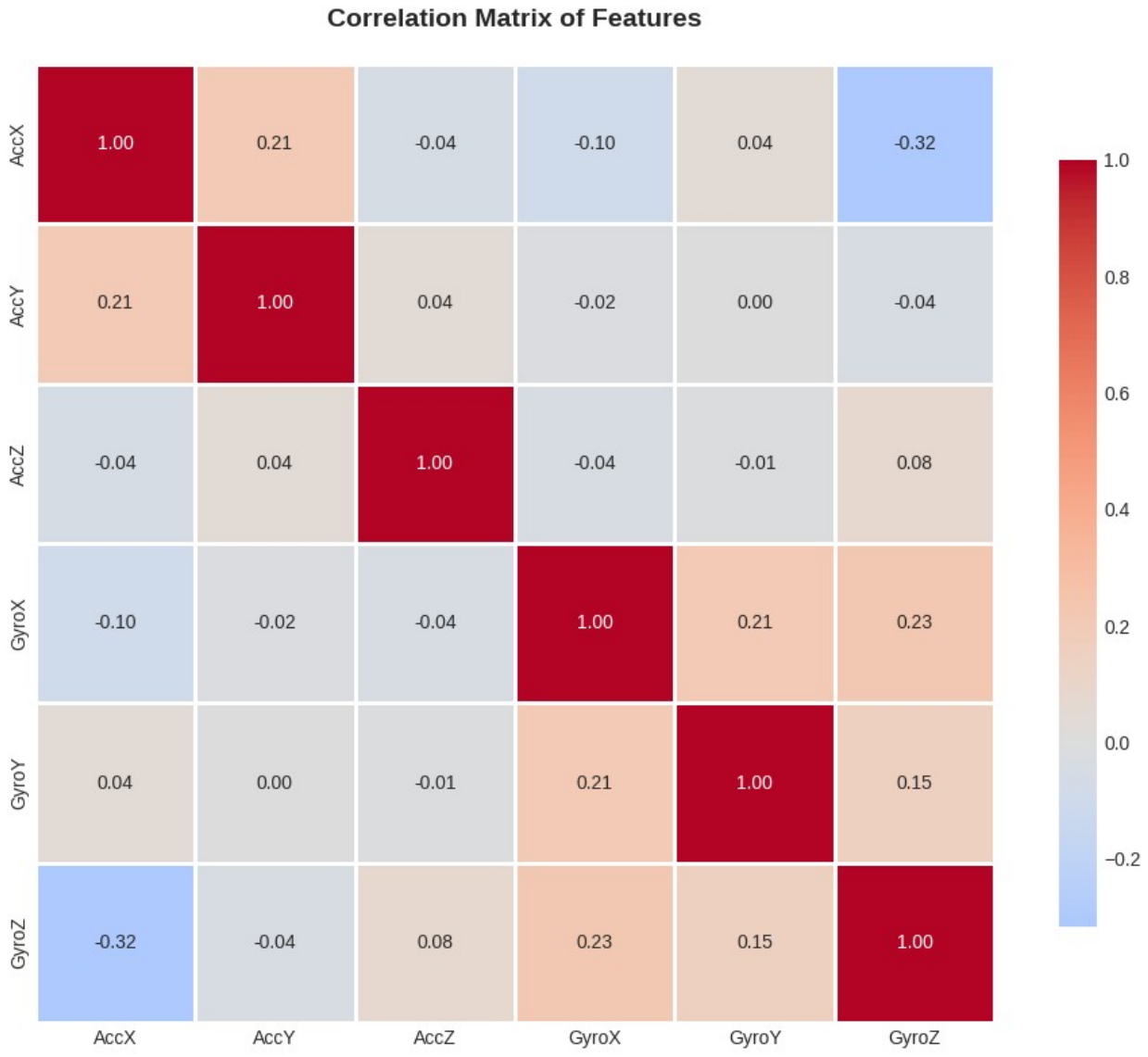
correlation_matrix = df[feature_list].corr()

fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, fmt='.2f',
cmap='coolwarm',
            center=0, square=True, linewidths=1, cbar_kws={"shrink":
0.8},
            ax=ax)
ax.set_title('Correlation Matrix of Features', fontsize=14,
fontweight='bold', pad=20)
plt.tight_layout()
plt.show()

print("\nKey Correlations:")
corr_pairs = []
for i in range(len(correlation_matrix.columns)):
    for j in range(i+1, len(correlation_matrix.columns)):
        corr_pairs.append((
```

```
            correlation_matrix.columns[i],
            correlation_matrix.columns[j],
            correlation_matrix.iloc[i, j]
        ))

corr_pairs.sort(key=lambda x: abs(x[2]), reverse=True)
for i, (feat1, feat2, corr) in enumerate(corr_pairs[:5]):
    print(f"{i+1}. {feat1} <-> {feat2}: {corr:.4f}")
```




```
Key Correlations:
1. AccX <-> GyroZ: -0.3162
2. GyroX <-> GyroZ: 0.2286
3. GyroX <-> GyroY: 0.2097
```

```
4. AccX <-> AccY: 0.2076
5. GyroY <-> GyroZ: 0.1541

print("FEATURE STATISTICS BY CLASS")


for feature in feature_list:
    print(f"\n{feature}:")
    for class_label in sorted(df['Class'].unique()):
        data = df[df['Class'] == class_label][feature]
        print(f"  {class_label:12s} - Mean: {data.mean():8.4f}, Std:
{data.std():8.4f}, "
              f"Min: {data.min():8.4f}, Max: {data.max():8.4f}")
```

```
FEATURE STATISTICS BY CLASS

AccX:
  AGGRESSIVE   - Mean:   0.0961, Std:   1.2183, Min:  -4.1329, Max:
4.9855
  NORMAL       - Mean:  -0.0180, Std:   0.8662, Min:  -2.8973, Max:
3.7570
  SLOW         - Mean:   0.0466, Std:   0.8590, Min:  -4.6365, Max:
3.2782

AccY:
  AGGRESSIVE   - Mean:  -0.1705, Std:   1.1387, Min:  -4.6998, Max:
4.2452
  NORMAL       - Mean:  -0.0277, Std:   0.8144, Min:  -3.5788, Max:
3.6508
  SLOW         - Mean:  -0.0335, Std:   0.7364, Min:  -2.8811, Max:
3.2739

AccZ:
  AGGRESSIVE   - Mean:  -0.0211, Std:   1.1292, Min:  -7.1440, Max:
5.1717
  NORMAL       - Mean:   0.0344, Std:   0.9645, Min:  -3.8255, Max:
4.9543
  SLOW         - Mean:   0.0093, Std:   0.8671, Min:  -3.6526, Max:
4.6649

GyroX:
  AGGRESSIVE   - Mean:   0.0002, Std:   0.0717, Min:  -0.5582, Max:
0.8493
  NORMAL       - Mean:   0.0019, Std:   0.0607, Min:  -0.2876, Max:
0.2818
  SLOW         - Mean:   0.0025, Std:   0.0681, Min:  -0.7518, Max:
0.6666

GyroY:
  AGGRESSIVE   - Mean:  -0.0029, Std:   0.1359, Min:  -1.2908, Max:
```

```
1.2553
  NORMAL       - Mean:  -0.0009, Std:   0.1175, Min:  -1.2908, Max:
0.6921
  SLOW         - Mean:  -0.0002, Std:   0.1254, Min:  -1.5870, Max:
1.6799

GyroZ:
  AGGRESSIVE   - Mean:   0.0111, Std:   0.1311, Min:  -0.8296, Max:
1.0060
  NORMAL       - Mean:   0.0095, Std:   0.1138, Min:  -0.9622, Max:
0.7311
  SLOW         - Mean:   0.0039, Std:   0.1030, Min:  -1.2365, Max:
1.1905
```

```
fig, axes = plt.subplots(3, 1, figsize=(16, 10))
fig.suptitle('Time Series Pattern of Accelerometer Data (Sample)',
             fontsize=14, fontweight='bold')

sample_normal = df[df['Class'] == 'NORMAL'].head(200)
sample_slow = df[df['Class'] == 'SLOW'].head(200)
sample_aggressive = df[df['Class'] == 'AGGRESSIVE'].head(200)

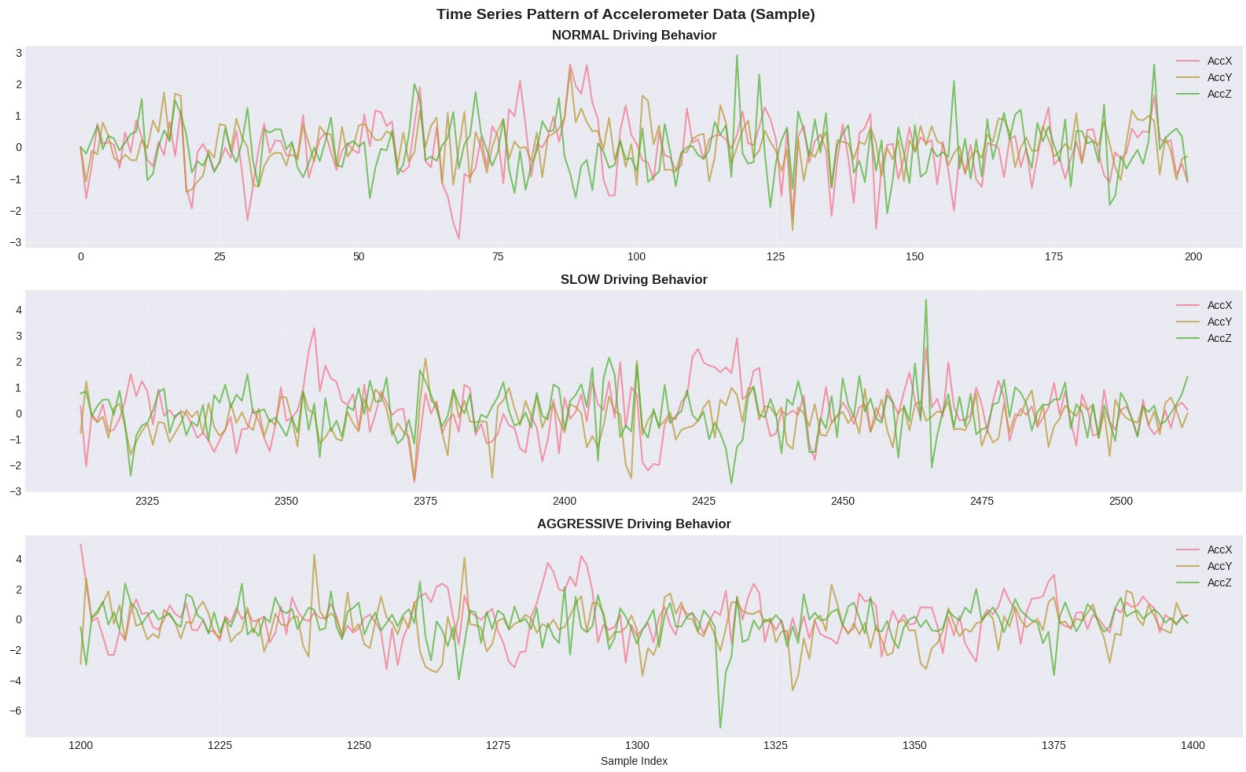
axes[0].plot(sample_normal.index, sample_normal['AccX'], label='AccX',
linewidth=1.5, alpha=0.7)
axes[0].plot(sample_normal.index, sample_normal['AccY'], label='AccY',
linewidth=1.5, alpha=0.7)
axes[0].plot(sample_normal.index, sample_normal['AccZ'], label='AccZ',
linewidth=1.5, alpha=0.7)
axes[0].set_title('NORMAL Driving Behavior', fontweight='bold')
axes[0].legend(loc='upper right')
axes[0].grid(alpha=0.3)

axes[1].plot(sample_slow.index, sample_slow['AccX'], label='AccX',
linewidth=1.5, alpha=0.7)
axes[1].plot(sample_slow.index, sample_slow['AccY'], label='AccY',
linewidth=1.5, alpha=0.7)
axes[1].plot(sample_slow.index, sample_slow['AccZ'], label='AccZ',
linewidth=1.5, alpha=0.7)
axes[1].set_title('SLOW Driving Behavior', fontweight='bold')
axes[1].legend(loc='upper right')
axes[1].grid(alpha=0.3)

axes[2].plot(sample_aggressive.index, sample_aggressive['AccX'],
label='AccX', linewidth=1.5, alpha=0.7)
axes[2].plot(sample_aggressive.index, sample_aggressive['AccY'],
label='AccY', linewidth=1.5, alpha=0.7)
axes[2].plot(sample_aggressive.index, sample_aggressive['AccZ'],
label='AccZ', linewidth=1.5, alpha=0.7)
axes[2].set_title('AGGRESSIVE Driving Behavior', fontweight='bold')
axes[2].set_xlabel('Sample Index')
```

```
axes[2].legend(loc='upper right')
axes[2].grid(alpha=0.3)

plt.tight_layout()
plt.show()
```



1. FEATURE ENGINEERING & JERK ANALYSIS

```
df_features = df.copy()

print("FEATURE ENGINEERING")


print("\n1. JERK CALCULATION (Sequential Differences)")


df_features['JerkX'] = df_features['AccX'].diff().fillna(0)
df_features['JerkY'] = df_features['AccY'].diff().fillna(0)
df_features['JerkZ'] = df_features['AccZ'].diff().fillna(0)

print(f"Created JerkX, JerkY, JerkZ features")
print(f"Jerk Statistics (sample):")
print(df_features[['JerkX', 'JerkY', 'JerkZ']].describe())
```

```
FEATURE ENGINEERING

1. JERK CALCULATION (Sequential Differences)
```

```
Created JerkX, JerkY, JerkZ features
Jerk Statistics (sample):
              JerkX         JerkY         JerkZ
count  3644.000000  3644.000000  3644.000000
mean     -0.000110     0.000111    -0.000116
std       1.072145     1.069747     1.376495
min      -5.706999    -4.541037    -6.669646
25%      -0.665266    -0.658628    -0.785964
50%       0.022132    -0.012872     0.006853
75%       0.676928     0.659883     0.801030
max       4.369801     6.718676     6.639951
```

```
print("\n2. MAGNITUDE-BASED FEATURES\n\n")



df_features['AccMagnitude'] = np.sqrt(df_features['AccX']**2 +
                                      df_features['AccY']**2 +
                                      df_features['AccZ']**2)

df_features['GyroMagnitude'] = np.sqrt(df_features['GyroX']**2 +
                                       df_features['GyroY']**2 +
                                       df_features['GyroZ']**2)

df_features['JerkMagnitude'] = np.sqrt(df_features['JerkX']**2 +
                                       df_features['JerkY']**2 +
                                       df_features['JerkZ']**2)

print(f"Created AccMagnitude, GyroMagnitude, JerkMagnitude features")
print(f"Magnitude Statistics:")
print(df_features[['AccMagnitude', 'GyroMagnitude',
'JerkMagnitude']].describe())
```

```
2. MAGNITUDE-BASED FEATURES


Created AccMagnitude, GyroMagnitude, JerkMagnitude features
Magnitude Statistics:
       AccMagnitude  GyroMagnitude  JerkMagnitude
count   3644.000000    3644.000000    3644.000000
mean       1.437810       0.130792       1.788648
std        0.835007       0.129403       0.994209
min        0.000000       0.003707       0.000000
25%        0.852041       0.058421       1.090478
50%        1.268063       0.096627       1.586346
75%        1.843263       0.157922       2.258135
max        7.451246       1.957027       7.146545
```

```
print("\n3. ROLLING WINDOW STATISTICS\n\n")
```

```
window_size = 30

df_features['AccX_rolling_mean'] =
df_features['AccX'].rolling(window=window_size, min_periods=1).mean()
df_features['AccY_rolling_mean'] =
df_features['AccY'].rolling(window=window_size, min_periods=1).mean()
df_features['AccZ_rolling_mean'] =
df_features['AccZ'].rolling(window=window_size, min_periods=1).mean()

df_features['AccX_rolling_std'] =
df_features['AccX'].rolling(window=window_size,
min_periods=1).std().fillna(0)
df_features['AccY_rolling_std'] =
df_features['AccY'].rolling(window=window_size,
min_periods=1).std().fillna(0)
df_features['AccZ_rolling_std'] =
df_features['AccZ'].rolling(window=window_size,
min_periods=1).std().fillna(0)

df_features['JerkX_rolling_mean'] =
df_features['JerkX'].rolling(window=window_size, min_periods=1).mean()
df_features['JerkY_rolling_mean'] =
df_features['JerkY'].rolling(window=window_size, min_periods=1).mean()
df_features['JerkZ_rolling_mean'] =
df_features['JerkZ'].rolling(window=window_size, min_periods=1).mean()


df_features['JerkX_rolling_max'] =
df_features['JerkX'].rolling(window=window_size, min_periods=1).max()
df_features['JerkY_rolling_max'] =
df_features['JerkY'].rolling(window=window_size, min_periods=1).max()
df_features['JerkZ_rolling_max'] =
df_features['JerkZ'].rolling(window=window_size, min_periods=1).max()

print(f"Created rolling mean and std for accelerations and jerk")
print(f"Total features created in rolling window: 12")


3. ROLLING WINDOW STATISTICS


Created rolling mean and std for accelerations and jerk
Total features created in rolling window: 12

print("\n4. STATISTICAL AGGREGATION FEATURES\n\n")

df_features['AccX_var'] = df_features['AccX'] ** 2
df_features['AccY_var'] = df_features['AccY'] ** 2
df_features['AccZ_var'] = df_features['AccZ'] ** 2
```

```
df_features['JerkX_var'] = df_features['JerkX'] ** 2
df_features['JerkY_var'] = df_features['JerkY'] ** 2
df_features['JerkZ_var'] = df_features['JerkZ'] ** 2

df_features['AbsAccX'] = abs(df_features['AccX'])
df_features['AbsAccY'] = abs(df_features['AccY'])
df_features['AbsAccZ'] = abs(df_features['AccZ'])
df_features['AbsJerkX'] = abs(df_features['JerkX'])
df_features['AbsJerkY'] = abs(df_features['JerkY'])
df_features['AbsJerkZ'] = abs(df_features['JerkZ'])

df_features['Acc_Energy'] = (df_features['AccX']**2 +
df_features['AccY']**2 + df_features['AccZ']**2) / 3
df_features['Jerk_Energy'] = (df_features['JerkX']**2 +
df_features['JerkY']**2 + df_features['JerkZ']**2) / 3

print(f"Created variance, absolute, and energy features")
print(f"Total new features: 18")


4. STATISTICAL AGGREGATION FEATURES


Created variance, absolute, and energy features
Total new features: 18

print("FEATURE ENGINEERING SUMMARY\n \n")

original_features = ['AccX', 'AccY', 'AccZ', 'GyroX', 'GyroY',
'GyroZ']
engineered_features = [col for col in df_features.columns if col not
in original_features + ['Class', 'Timestamp']]

print(f"\nOriginal Features: {len(original_features)}")
print(f"  {original_features}")
print(f"\nEngineered Features: {len(engineered_features)}")

jerk_features = [f for f in engineered_features if 'Jerk' in f]
magnitude_features = [f for f in engineered_features if 'Magnitude' in
f]
rolling_features = [f for f in engineered_features if 'rolling' in f]
other_features = [f for f in engineered_features if f not in
jerk_features + magnitude_features + rolling_features]

print(f"\n  Jerk Features ({len(jerk_features)}): {jerk_features}")
print(f"  Magnitude Features ({len(magnitude_features)}):
{magnitude_features}")
print(f"  Rolling Window Features ({len(rolling_features)}):
{rolling_features}")
print(f"  Other Features ({len(other_features)}): {other_features}")
```

```
print(f"\nTOTAL FEATURES FOR MODELING: {len(original_features) +
len(engineered_features)}")
print(f"\nDataset shape after feature engineering:
{df_features.shape}")
```

```
FEATURE ENGINEERING SUMMARY



Original Features: 6
  ['AccX', 'AccY', 'AccZ', 'GyroX', 'GyroY', 'GyroZ']

Engineered Features: 32

  Jerk Features (17): ['JerkX', 'JerkY', 'JerkZ', 'JerkMagnitude',
'JerkX_rolling_mean', 'JerkY_rolling_mean', 'JerkZ_rolling_mean',
'JerkX_rolling_max', 'JerkY_rolling_max', 'JerkZ_rolling_max',
'JerkX_var', 'JerkY_var', 'JerkZ_var', 'AbsJerkX', 'AbsJerkY',
'AbsJerkZ', 'Jerk_Energy']
  Magnitude Features (3): ['AccMagnitude', 'GyroMagnitude',
'JerkMagnitude']
  Rolling Window Features (12): ['AccX_rolling_mean',
'AccY_rolling_mean', 'AccZ_rolling_mean', 'AccX_rolling_std',
'AccY_rolling_std', 'AccZ_rolling_std', 'JerkX_rolling_mean',
'JerkY_rolling_mean', 'JerkZ_rolling_mean', 'JerkX_rolling_max',
'JerkY_rolling_max', 'JerkZ_rolling_max']
  Other Features (7): ['AccX_var', 'AccY_var', 'AccZ_var', 'AbsAccX',
'AbsAccY', 'AbsAccZ', 'Acc_Energy']

TOTAL FEATURES FOR MODELING: 38

Dataset shape after feature engineering: (3644, 40)
```

```
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
fig.suptitle('Jerk Distribution by Driving Behavior Class',
fontsize=14, fontweight='bold')

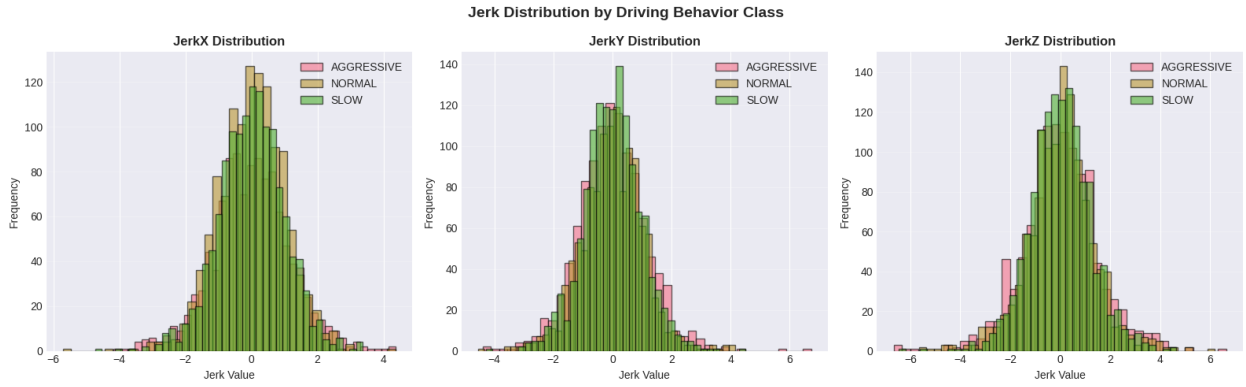
jerk_cols = ['JerkX', 'JerkY', 'JerkZ']
for idx, col in enumerate(jerk_cols):
    for class_label in sorted(df_features['Class'].unique()):
        data = df_features[df_features['Class'] == class_label][col]
        axes[idx].hist(data, alpha=0.6, label=class_label, bins=40,
edgecolor='black')

    axes[idx].set_title(f'{col} Distribution', fontweight='bold')
    axes[idx].set_xlabel('Jerk Value')
    axes[idx].set_ylabel('Frequency')
    axes[idx].legend()
    axes[idx].grid(axis='y', alpha=0.3)

plt.tight_layout()
```

```
plt.show()

print("✓ Jerk features show distinct patterns across classes!")
```




```
✓ Jerk features show distinct patterns across classes!
```

1. DATA PREPARATION FOR MODELING

```
print("DATA PREPARATION FOR MODELING\n \n")


feature_cols = [col for col in df_features.columns if col not in
['Class', 'Timestamp']]

X = df_features[feature_cols].copy()
y = df_features['Class'].copy()

print(f"\nFeature Matrix Shape: {X.shape}")
print(f"Target Variable Shape: {y.shape}")
print(f"\nNumber of Features: {X.shape[1]}")
print(f"Feature Names:\n{X.columns.tolist()}")

print(f"\nMissing Values in Features:")
missing = X.isnull().sum()
if missing.sum() > 0:
    print(missing[missing > 0])
    X = X.fillna(0)
    print("✓ Missing values filled with 0")
else:
    print("✓ No missing values!")

DATA PREPARATION FOR MODELING



Feature Matrix Shape: (3644, 38)
Target Variable Shape: (3644,)
```

```
Number of Features: 38
Feature Names:
['AccX', 'AccY', 'AccZ', 'GyroX', 'GyroY', 'GyroZ', 'JerkX', 'JerkY',
'JerkZ', 'AccMagnitude', 'GyroMagnitude', 'JerkMagnitude',
'AccX_rolling_mean', 'AccY_rolling_mean', 'AccZ_rolling_mean',
'AccX_rolling_std', 'AccY_rolling_std', 'AccZ_rolling_std',
'JerkX_rolling_mean', 'JerkY_rolling_mean', 'JerkZ_rolling_mean',
'JerkX_rolling_max', 'JerkY_rolling_max', 'JerkZ_rolling_max',
'AccX_var', 'AccY_var', 'AccZ_var', 'JerkX_var', 'JerkY_var',
'JerkZ_var', 'AbsAccX', 'AbsAccY', 'AbsAccZ', 'AbsJerkX', 'AbsJerkY',
'AbsJerkZ', 'Acc_Energy', 'Jerk_Energy']

Missing Values in Features:
✓ No missing values!
```

```
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)

print("\nTarget Variable Encoding:\n")
for idx, class_label in enumerate(label_encoder.classes_):
    print(f"  {class_label} → {idx}")

print(f"\nEncoded Target Distribution:\n")
unique, counts = np.unique(y_encoded, return_counts=True)
for u, c in zip(unique, counts):
    class_name = label_encoder.inverse_transform([u])[0]
    print(f"  {class_name}: {c} samples ({c/len(y_encoded)*100:.2f}
%)")
```

```
Target Variable Encoding:

  AGGRESSIVE → 0
  NORMAL → 1
  SLOW → 2

Encoded Target Distribution:

  AGGRESSIVE: 1113 samples (30.54%)
  NORMAL: 1200 samples (32.93%)
  SLOW: 1331 samples (36.53%)
```

```
print("\nTrain-Test Split (80-20):")
X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
)

print(f"Training Set Size: {X_train.shape[0]} samples
({X_train.shape[0]/len(X)*100:.2f}%)")
print(f"Test Set Size: {X_test.shape[0]} samples
({X_test.shape[0]/len(X)*100:.2f}%)")
```

```
print(f"\nTraining Set Class Distribution:")
for u, c in zip(*np.unique(y_train, return_counts=True)):
    class_name = label_encoder.inverse_transform([u])[0]
    print(f"  {class_name}: {c} samples")
print(f"\nTest Set Class Distribution:")
for u, c in zip(*np.unique(y_test, return_counts=True)):
    class_name = label_encoder.inverse_transform([u])[0]
    print(f"  {class_name}: {c} samples")
```

```
Train-Test Split (80-20):
Training Set Size: 2915 samples (79.99%)
Test Set Size: 729 samples (20.01%)

Training Set Class Distribution:
  AGGRESSIVE: 890 samples
  NORMAL: 960 samples
  SLOW: 1065 samples

Test Set Class Distribution:
  AGGRESSIVE: 223 samples
  NORMAL: 240 samples
  SLOW: 266 samples
```

```
print("\nFeature Scaling:")
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("✓ Features scaled using StandardScaler")
print(f"\nScaled Training Data Statistics:")
print(f"  Mean: {X_train_scaled.mean():.6f}")
print(f"  Std Dev: {X_train_scaled.std():.6f}")
print(f"  Min: {X_train_scaled.min():.6f}")
print(f"  Max: {X_train_scaled.max():.6f}")
```

```
Feature Scaling:
✓ Features scaled using StandardScaler

Scaled Training Data Statistics:
  Mean: -0.000000
  Std Dev: 1.000000
  Min: -16.793035
  Max: 23.610539
```

6. MODEL TRAINING & EVALUATION

```
models_dict = {}
results_dict = {}
```

```
print("TRAINING MULTIPLE SCIKIT-LEARN MODELS")

print("\n1. LOGISTIC REGRESSION\n")

lr_model = LogisticRegression(max_iter=1000, random_state=42,
multi_class='multinomial')
lr_model.fit(X_train_scaled, y_train)
y_pred_lr = lr_model.predict(X_test_scaled)
y_pred_proba_lr = lr_model.predict_proba(X_test_scaled)

models_dict['Logistic Regression'] = lr_model
results_dict['Logistic Regression'] = {
    'predictions': y_pred_lr,
    'probabilities': y_pred_proba_lr,
    'accuracy': accuracy_score(y_test, y_pred_lr),
    'precision': precision_score(y_test, y_pred_lr,
average='weighted'),
    'recall': recall_score(y_test, y_pred_lr, average='weighted'),
    'f1': f1_score(y_test, y_pred_lr, average='weighted')
}

print(f"Accuracy: {results_dict['Logistic Regression']
['accuracy']:.4f}")
print(f"Precision (Weighted): {results_dict['Logistic Regression']
['precision']:.4f}")
print(f"Recall (Weighted): {results_dict['Logistic Regression']
['recall']:.4f}")
print(f"F1-Score (Weighted): {results_dict['Logistic Regression']
['f1']:.4f}")
```

```
TRAINING MULTIPLE SCIKIT-LEARN MODELS

1. LOGISTIC REGRESSION

Accuracy: 0.5926
Precision (Weighted): 0.5940
Recall (Weighted): 0.5926
F1-Score (Weighted): 0.5893
```

```
print("\n2. RANDOM FOREST CLASSIFIER\n \n")

rf_model = RandomForestClassifier(n_estimators=100, max_depth=20,
                                  random_state=42, n_jobs=-1,
min_samples_split=5)
rf_model.fit(X_train_scaled, y_train)
y_pred_rf = rf_model.predict(X_test_scaled)
y_pred_proba_rf = rf_model.predict_proba(X_test_scaled)

models_dict['Random Forest'] = rf_model
results_dict['Random Forest'] = {
```

```
    'predictions': y_pred_rf,
    'probabilities': y_pred_proba_rf,
    'accuracy': accuracy_score(y_test, y_pred_rf),
    'precision': precision_score(y_test, y_pred_rf,
average='weighted'),
    'recall': recall_score(y_test, y_pred_rf, average='weighted'),
    'f1': f1_score(y_test, y_pred_rf, average='weighted')
}

print(f"Accuracy: {results_dict['Random Forest']['accuracy']:.4f}")
print(f"Precision (Weighted): {results_dict['Random Forest']
['precision']:.4f}")
print(f"Recall (Weighted): {results_dict['Random Forest']
['recall']:.4f}")
print(f"F1-Score (Weighted): {results_dict['Random Forest']
['f1']:.4f}")

feature_importance = pd.DataFrame({
    'feature': feature_cols,
    'importance': rf_model.feature_importances_
}).sort_values('importance', ascending=False)

print(f"\nTop 10 Most Important Features:")
print(feature_importance.head(10).to_string(index=False))


2. RANDOM FOREST CLASSIFIER


Accuracy: 0.9588
Precision (Weighted): 0.9594
Recall (Weighted): 0.9588
F1-Score (Weighted): 0.9589

Top 10 Most Important Features:
          feature  importance
 AccY_rolling_std    0.155708
 AccX_rolling_std    0.102984
JerkX_rolling_max    0.095911
JerkY_rolling_max    0.081600
JerkZ_rolling_max    0.077124
 AccZ_rolling_std    0.062322
AccY_rolling_mean    0.058832
AccZ_rolling_mean    0.049814
AccX_rolling_mean    0.043193
            GyroZ    0.018691

print("\n3. GRADIENT BOOSTING CLASSIFIER\n \n")

gb_model = GradientBoostingClassifier(n_estimators=100,
```

```
learning_rate=0.1,
                                        max_depth=7, random_state=42)
gb_model.fit(X_train_scaled, y_train)
y_pred_gb = gb_model.predict(X_test_scaled)
y_pred_proba_gb = gb_model.predict_proba(X_test_scaled)

models_dict['Gradient Boosting'] = gb_model
results_dict['Gradient Boosting'] = {
    'predictions': y_pred_gb,
    'probabilities': y_pred_proba_gb,
    'accuracy': accuracy_score(y_test, y_pred_gb),
    'precision': precision_score(y_test, y_pred_gb,
average='weighted'),
    'recall': recall_score(y_test, y_pred_gb, average='weighted'),
    'f1': f1_score(y_test, y_pred_gb, average='weighted')
}

print(f"Accuracy: {results_dict['Gradient Boosting']
['accuracy']:.4f}")
print(f"Precision (Weighted): {results_dict['Gradient Boosting']
['precision']:.4f}")
print(f"Recall (Weighted): {results_dict['Gradient Boosting']
['recall']:.4f}")
print(f"F1-Score (Weighted): {results_dict['Gradient Boosting']
['f1']:.4f}")


3. GRADIENT BOOSTING CLASSIFIER


Accuracy: 0.9918
Precision (Weighted): 0.9918
Recall (Weighted): 0.9918
F1-Score (Weighted): 0.9917

print("\n4. SUPPORT VECTOR MACHINE (SVM)\n \n")

svm_model = SVC(kernel='rbf', C=1.0, gamma='scale', probability=True,
random_state=42)
svm_model.fit(X_train_scaled, y_train)
y_pred_svm = svm_model.predict(X_test_scaled)
y_pred_proba_svm = svm_model.predict_proba(X_test_scaled)

models_dict['SVM'] = svm_model
results_dict['SVM'] = {
    'predictions': y_pred_svm,
    'probabilities': y_pred_proba_svm,
    'accuracy': accuracy_score(y_test, y_pred_svm),
    'precision': precision_score(y_test, y_pred_svm,
average='weighted'),
```

```
    'recall': recall_score(y_test, y_pred_svm, average='weighted'),
    'f1': f1_score(y_test, y_pred_svm, average='weighted')
}

print(f"Accuracy: {results_dict['SVM']['accuracy']:.4f}")
print(f"Precision (Weighted): {results_dict['SVM']['precision']:.4f}")
print(f"Recall (Weighted): {results_dict['SVM']['recall']:.4f}")
print(f"F1-Score (Weighted): {results_dict['SVM']['f1']:.4f}")


4. SUPPORT VECTOR MACHINE (SVM)


Accuracy: 0.6255
Precision (Weighted): 0.6245
Recall (Weighted): 0.6255
F1-Score (Weighted): 0.6233

print("\n5. K-NEAREST NEIGHBORS (KNN)\n\n")

knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train_scaled, y_train)
y_pred_knn = knn_model.predict(X_test_scaled)
y_pred_proba_knn = knn_model.predict_proba(X_test_scaled)

models_dict['KNN'] = knn_model
results_dict['KNN'] = {
    'predictions': y_pred_knn,
    'probabilities': y_pred_proba_knn,
    'accuracy': accuracy_score(y_test, y_pred_knn),
    'precision': precision_score(y_test, y_pred_knn,
average='weighted'),
    'recall': recall_score(y_test, y_pred_knn, average='weighted'),
    'f1': f1_score(y_test, y_pred_knn, average='weighted')
}

print(f"Accuracy: {results_dict['KNN']['accuracy']:.4f}")
print(f"Precision (Weighted): {results_dict['KNN']['precision']:.4f}")
print(f"Recall (Weighted): {results_dict['KNN']['recall']:.4f}")
print(f"F1-Score (Weighted): {results_dict['KNN']['f1']:.4f}")


5. K-NEAREST NEIGHBORS (KNN)


Accuracy: 0.6036
Precision (Weighted): 0.6161
Recall (Weighted): 0.6036
F1-Score (Weighted): 0.6077
```

```
print("\n6. DECISION TREE CLASSIFIER\n \n")

dt_model = DecisionTreeClassifier(max_depth=15, random_state=42,
min_samples_split=5)
dt_model.fit(X_train_scaled, y_train)
y_pred_dt = dt_model.predict(X_test_scaled)
y_pred_proba_dt = dt_model.predict_proba(X_test_scaled)

models_dict['Decision Tree'] = dt_model
results_dict['Decision Tree'] = {
    'predictions': y_pred_dt,
    'probabilities': y_pred_proba_dt,
    'accuracy': accuracy_score(y_test, y_pred_dt),
    'precision': precision_score(y_test, y_pred_dt,
average='weighted'),
    'recall': recall_score(y_test, y_pred_dt, average='weighted'),
    'f1': f1_score(y_test, y_pred_dt, average='weighted')
}

print(f"Accuracy: {results_dict['Decision Tree']['accuracy']:.4f}")
print(f"Precision (Weighted): {results_dict['Decision Tree']
['precision']:.4f}")
print(f"Recall (Weighted): {results_dict['Decision Tree']
['recall']:.4f}")
print(f"F1-Score (Weighted): {results_dict['Decision Tree']
['f1']:.4f}")
```

```
6. DECISION TREE CLASSIFIER


Accuracy: 0.8944
Precision (Weighted): 0.8958
Recall (Weighted): 0.8944
F1-Score (Weighted): 0.8944
```

```
print("\n7. GAUSSIAN NAIVE BAYES\n \n")

nb_model = GaussianNB()
nb_model.fit(X_train_scaled, y_train)
y_pred_nb = nb_model.predict(X_test_scaled)
y_pred_proba_nb = nb_model.predict_proba(X_test_scaled)

models_dict['Naive Bayes'] = nb_model
results_dict['Naive Bayes'] = {
    'predictions': y_pred_nb,
    'probabilities': y_pred_proba_nb,
    'accuracy': accuracy_score(y_test, y_pred_nb),
    'precision': precision_score(y_test, y_pred_nb,
average='weighted'),
```

```
    'recall': recall_score(y_test, y_pred_nb, average='weighted'),
    'f1': f1_score(y_test, y_pred_nb, average='weighted')
}

print(f"Accuracy: {results_dict['Naive Bayes']['accuracy']:.4f}")
print(f"Precision (Weighted): {results_dict['Naive Bayes']
['precision']:.4f}")
print(f"Recall (Weighted): {results_dict['Naive Bayes']
['recall']:.4f}")
print(f"F1-Score (Weighted): {results_dict['Naive Bayes']['f1']:.4f}")


7. GAUSSIAN NAIVE BAYES


Accuracy: 0.4911
Precision (Weighted): 0.4791
Recall (Weighted): 0.4911
F1-Score (Weighted): 0.4595

print("\n8. ADABOOST CLASSIFIER\n \n")

ada_model = AdaBoostClassifier(n_estimators=50, learning_rate=1.0,
random_state=42)
ada_model.fit(X_train_scaled, y_train)
y_pred_ada = ada_model.predict(X_test_scaled)
y_pred_proba_ada = ada_model.predict_proba(X_test_scaled)

models_dict['AdaBoost'] = ada_model
results_dict['AdaBoost'] = {
    'predictions': y_pred_ada,
    'probabilities': y_pred_proba_ada,
    'accuracy': accuracy_score(y_test, y_pred_ada),
    'precision': precision_score(y_test, y_pred_ada,
average='weighted'),
    'recall': recall_score(y_test, y_pred_ada, average='weighted'),
    'f1': f1_score(y_test, y_pred_ada, average='weighted')
}

print(f"Accuracy: {results_dict['AdaBoost']['accuracy']:.4f}")
print(f"Precision (Weighted): {results_dict['AdaBoost']
['precision']:.4f}")
print(f"Recall (Weighted): {results_dict['AdaBoost']['recall']:.4f}")
print(f"F1-Score (Weighted): {results_dict['AdaBoost']['f1']:.4f}")


8. ADABOOST CLASSIFIER


Accuracy: 0.6063
Precision (Weighted): 0.6094
```

```
Recall (Weighted): 0.6063
F1-Score (Weighted): 0.6048
```

1. COMPREHENSIVE MODEL COMPARISON & EVALUATION

```
print("MODEL PERFORMANCE COMPARISON\n \n")


comparison_df = pd.DataFrame(results_dict).T[['accuracy', 'precision',
'recall', 'f1']].round(4)
comparison_df = comparison_df.sort_values('f1', ascending=False)

print(f"\nAll Models Performance Metrics:")
print(comparison_df)

print(f"\nBest Model by F1-Score: {comparison_df['f1'].idxmax()}
({comparison_df['f1'].max():.4f})")
print(f"Best Model by Accuracy: {comparison_df['accuracy'].idxmax()}
({comparison_df['accuracy'].max():.4f})")
```

```
MODEL PERFORMANCE COMPARISON



All Models Performance Metrics:
                     accuracy precision    recall        f1
Gradient Boosting     0.99177  0.991806   0.99177  0.991749
Random Forest        0.958848  0.959403  0.958848  0.958942
Decision Tree        0.894376  0.895771  0.894376   0.89443
SVM                  0.625514  0.624466  0.625514   0.62334
KNN                  0.603567  0.616056  0.603567  0.607747
AdaBoost              0.60631  0.609436   0.60631  0.604815
Logistic Regression  0.592593  0.594034  0.592593  0.589333
Naive Bayes          0.491084  0.479123  0.491084  0.459544

Best Model by F1-Score: Gradient Boosting (0.9917)
Best Model by Accuracy: Gradient Boosting (0.9918)
```

```
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
fig.suptitle('Model Performance Comparison', fontsize=16,
fontweight='bold')

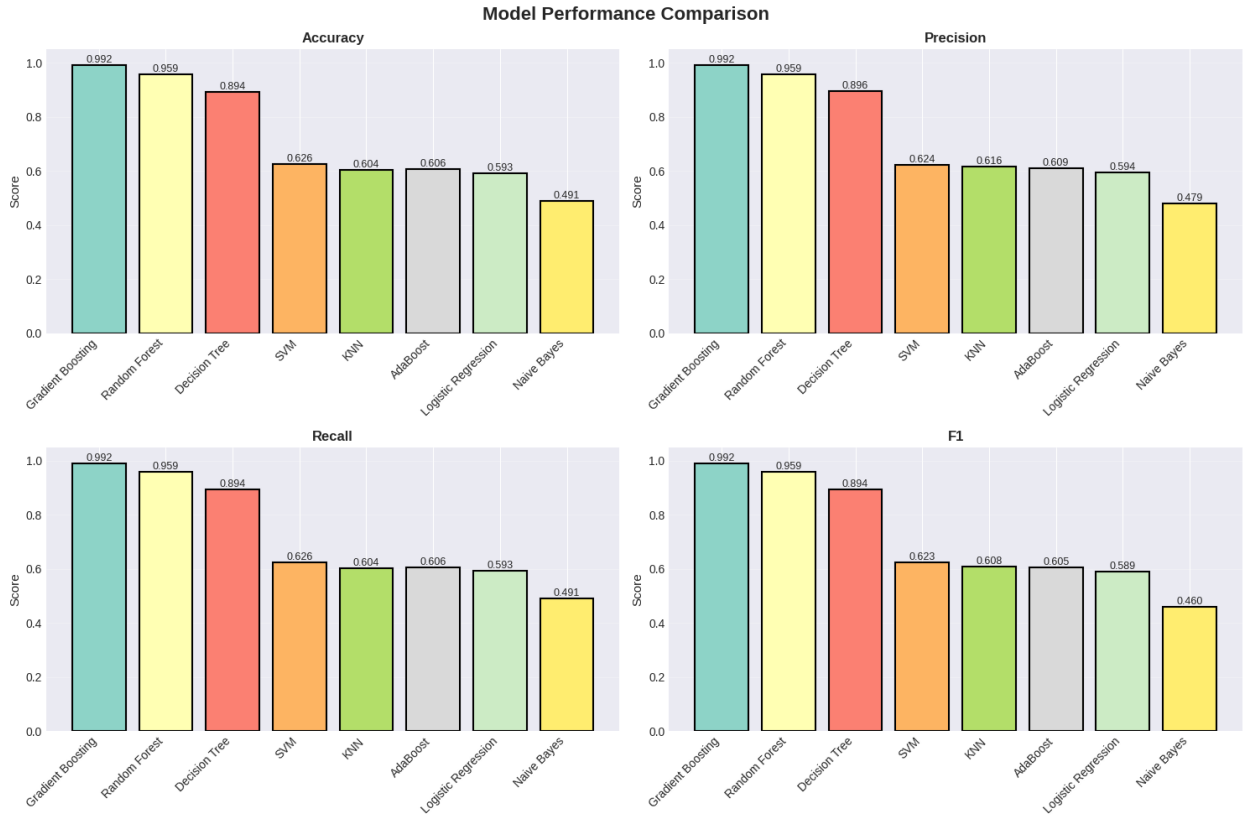
metrics = ['accuracy', 'precision', 'recall', 'f1']
colors_models = plt.cm.Set3(np.linspace(0, 1, len(comparison_df)))

for idx, metric in enumerate(metrics):
    ax = axes[idx // 2, idx % 2]
    values = comparison_df[metric].values
    bars = ax.bar(range(len(comparison_df)), values,
color=colors_models, edgecolor='black', linewidth=1.5)
```

```
    ax.set_xticks(range(len(comparison_df)))
    ax.set_xticklabels(comparison_df.index, rotation=45, ha='right')
    ax.set_title(f'{metric.capitalize()}', fontweight='bold')
    ax.set_ylabel('Score', fontsize=11)
    ax.set_ylim([0, 1.05])
    ax.grid(axis='y', alpha=0.3)

    for bar in bars:
        height = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2., height,
                f'{height:.3f}', ha='center', va='bottom', fontsize=9)

plt.tight_layout()
plt.show()
```




```
best_model_name = comparison_df['f1'].idxmax()
best_model = models_dict[best_model_name]
best_predictions = results_dict[best_model_name]['predictions']

print("\n \n")
print(f"DETAILED EVALUATION - BEST MODEL: {best_model_name}")
print("\n \n")

print(f"\nClassification Report:\n")
```

```
print(classification_report(y_test, best_predictions,
                            target_names=label_encoder.classes_,
                            digits=4))

print(f"\nConfusion Matrix:")
cm = confusion_matrix(y_test, best_predictions)
print(cm)
print(f"\nConfusion Matrix (Decoded):")
cm_df = pd.DataFrame(cm, index=label_encoder.classes_,
columns=label_encoder.classes_)
print(cm_df)
```

```
DETAILED EVALUATION - BEST MODEL: Gradient Boosting
```

```
Classification Report:

              precision    recall  f1-score   support

  AGGRESSIVE     0.9954    0.9776    0.9864       223
      NORMAL     0.9877    1.0000    0.9938       240
        SLOW     0.9925    0.9962    0.9944       266

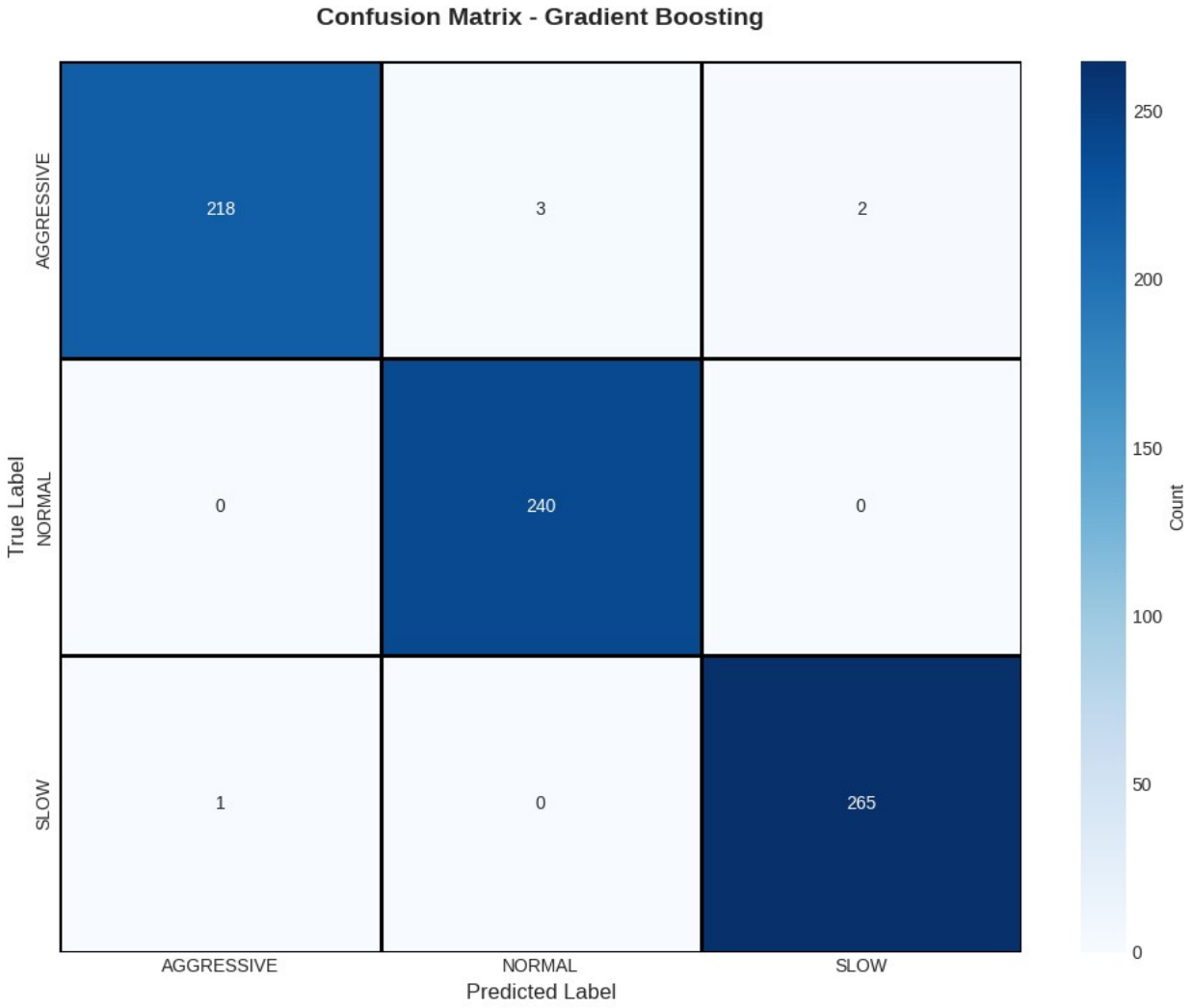
    accuracy                         0.9918       729
   macro avg     0.9919    0.9913    0.9915       729
weighted avg     0.9918    0.9918    0.9917       729


Confusion Matrix:
[[218   3   2]
 [  0 240   0]
 [  1   0 265]]

Confusion Matrix (Decoded):
            AGGRESSIVE  NORMAL  SLOW
AGGRESSIVE         218       3     2
NORMAL               0     240     0
SLOW                 1       0   265
```

```
fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=label_encoder.classes_,
            yticklabels=label_encoder.classes_,
            cbar_kws={'label': 'Count'},
            linewidths=2, linecolor='black', ax=ax)
ax.set_title(f'Confusion Matrix - {best_model_name}', fontsize=14,
```

```
fontweight='bold', pad=20)
ax.set_ylabel('True Label', fontsize=12)
ax.set_xlabel('Predicted Label', fontsize=12)
plt.tight_layout()
plt.show()
```




```
print("\n \n")
print(f"CROSS-VALIDATION ANALYSIS - {best_model_name}")
print("\n \n")


kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(best_model, X_train_scaled, y_train,
cv=kfold, scoring='f1_weighted')

print(f"\n5-Fold Cross-Validation F1-Scores:")
for i, score in enumerate(cv_scores, 1):
```

```
    print(f"  Fold {i}: {score:.4f}")

print(f"\nMean CV Score: {cv_scores.mean():.4f}")
print(f"Std Dev: {cv_scores.std():.4f}")
print(f"Confidence Interval (95%): [{cv_scores.mean() -
1.96*cv_scores.std():.4f}, {cv_scores.mean() +
1.96*cv_scores.std():.4f}]")



CROSS-VALIDATION ANALYSIS - Gradient Boosting



5-Fold Cross-Validation F1-Scores:
  Fold 1: 0.9811
  Fold 2: 0.9777
  Fold 3: 0.9845
  Fold 4: 0.9949
  Fold 5: 0.9914

Mean CV Score: 0.9859
Std Dev: 0.0064
Confidence Interval (95%): [0.9735, 0.9984]

if best_model_name in ['Random Forest', 'Gradient Boosting', 'Decision
Tree', 'AdaBoost']:
    print("\n \n")
    print(f"FEATURE IMPORTANCE - {best_model_name}")


    feature_imp = pd.DataFrame({
        'Feature': feature_cols,
        'Importance': best_model.feature_importances_
    }).sort_values('Importance', ascending=False)

    print(f"\nTop 15 Most Important Features:")
    print(feature_imp.head(15).to_string(index=False))

    fig, ax = plt.subplots(figsize=(12, 8))
    top_features = feature_imp.head(15)
    bars = ax.barh(range(len(top_features)),
top_features['Importance'].values,
                   color=plt.cm.viridis(np.linspace(0, 1,
len(top_features))),
                   edgecolor='black', linewidth=1.5)
    ax.set_yticks(range(len(top_features)))
    ax.set_yticklabels(top_features['Feature'].values)
    ax.set_xlabel('Importance Score', fontsize=12, fontweight='bold')
```

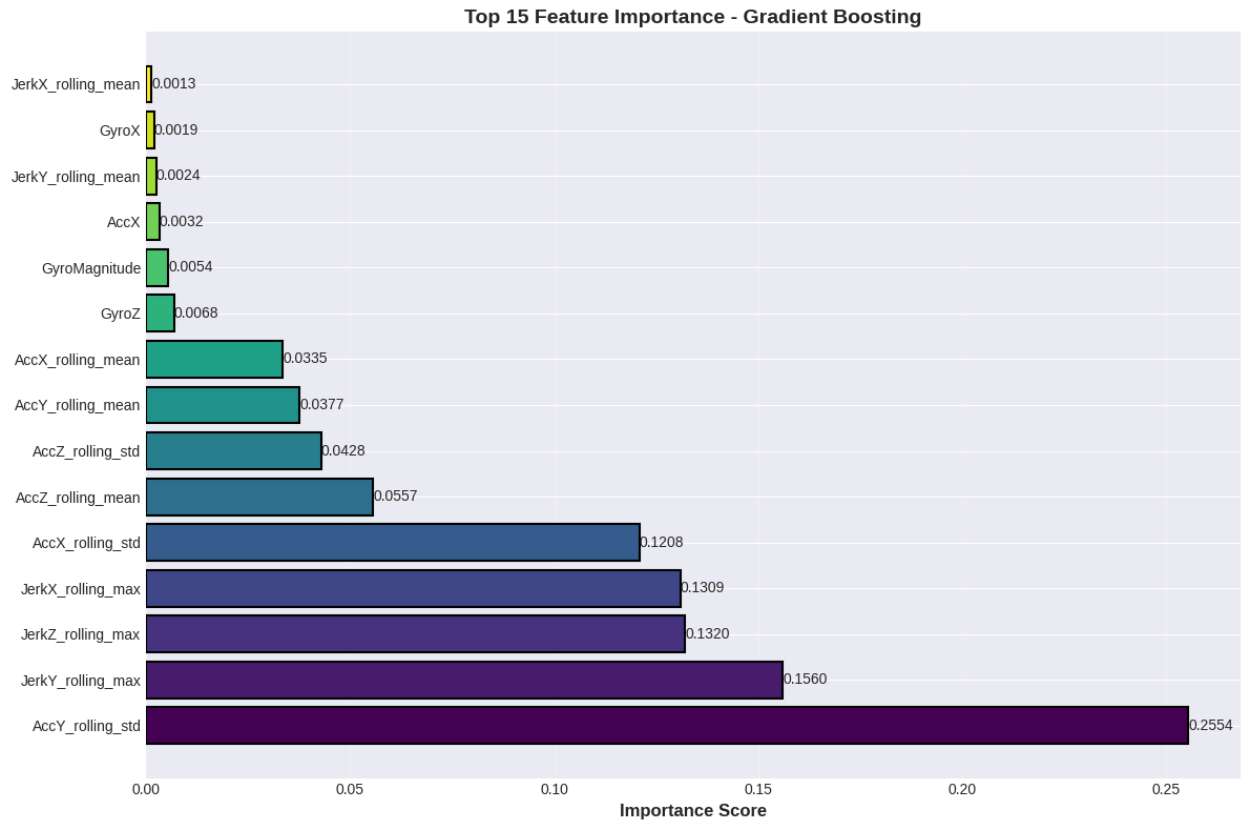
```
    ax.set_title(f'Top 15 Feature Importance - {best_model_name}',
fontsize=14, fontweight='bold')
    ax.grid(axis='x', alpha=0.3)

    for i, bar in enumerate(bars):
        width = bar.get_width()
        ax.text(width, bar.get_y() + bar.get_height()/2.,
                f'{width:.4f}', ha='left', va='center', fontsize=10)

    plt.tight_layout()
    plt.show()
```

```
FEATURE IMPORTANCE - Gradient Boosting

Top 15 Most Important Features:
           Feature  Importance
  AccY_rolling_std    0.255418
 JerkY_rolling_max    0.156008
 JerkZ_rolling_max    0.132036
 JerkX_rolling_max    0.130890
  AccX_rolling_std    0.120834
 AccZ_rolling_mean    0.055665
  AccZ_rolling_std    0.042826
 AccY_rolling_mean    0.037653
 AccX_rolling_mean    0.033461
             GyroZ    0.006771
     GyroMagnitude    0.005395
              AccX    0.003184
JerkY_rolling_mean    0.002385
             GyroX    0.001914
JerkX_rolling_mean    0.001250
```



1. MODEL OPTIMIZATION & HYPERPARAMETER TUNING

```
print("\n")
print(f"HYPERPARAMETER TUNING - {best_model_name}")
print("\n \n")


if best_model_name == 'Random Forest':
    print("\nTuning Random Forest hyperparameters...")
    param_grid = {
        'n_estimators': [100, 200],
        'max_depth': [15, 20, 25],
        'min_samples_split': [3, 5],
        'min_samples_leaf': [1, 2]
    }

elif best_model_name == 'Gradient Boosting':
    print("\nTuning Gradient Boosting hyperparameters...")
    param_grid = {
        'n_estimators': [100, 150],
        'learning_rate': [0.05, 0.1, 0.15],
        'max_depth': [5, 7, 10]
    }

elif best_model_name == 'SVM':
```

```
    print("\nTuning SVM hyperparameters...")
    param_grid = {
        'C': [0.1, 1, 10],
        'gamma': ['scale', 'auto'],
        'kernel': ['rbf', 'poly']
    }
else:
    print("\nUsing default parameters (model not tuned)")
    param_grid = None

if param_grid:
    print(f"\nGrid Search Configuration:")
    print(f"Parameter combinations to test: {np.prod([len(v) for v in
param_grid.values()])}")

    grid_search = GridSearchCV(best_model, param_grid, cv=3,
scoring='f1_weighted', n_jobs=-1, verbose=1)
    grid_search.fit(X_train_scaled, y_train)

    print(f"\nBest Parameters: {grid_search.best_params_}")
    print(f"Best Cross-Validation Score:
{grid_search.best_score_:.4f}")

    best_optimized_model = grid_search.best_estimator_
    y_pred_optimized = best_optimized_model.predict(X_test_scaled)
    optimized_accuracy = accuracy_score(y_test, y_pred_optimized)
    optimized_f1 = f1_score(y_test, y_pred_optimized,
average='weighted')

    print(f"\nOptimized Model Test Performance:")
    print(f"  Accuracy: {optimized_accuracy:.4f}")
    print(f"  F1-Score: {optimized_f1:.4f}")

    original_accuracy = results_dict[best_model_name]['accuracy']
    original_f1 = results_dict[best_model_name]['f1']

    print(f"\nImprovement:")
    print(f"  Accuracy: {optimized_accuracy - original_accuracy:.4f}")
    print(f"  F1-Score: {optimized_f1 - original_f1:.4f}")

    best_model = best_optimized_model
    best_predictions = y_pred_optimized


HYPERPARAMETER TUNING - Gradient Boosting



Tuning Gradient Boosting hyperparameters...
```

```
Grid Search Configuration:
Parameter combinations to test: 18
Fitting 3 folds for each of 18 candidates, totalling 54 fits

Best Parameters: {'learning_rate': 0.15, 'max_depth': 7,
'n_estimators': 150}
Best Cross-Validation Score: 0.9876

Optimized Model Test Performance:
  Accuracy: 0.9959
  F1-Score: 0.9959

Improvement:
  Accuracy: 0.0041
  F1-Score: 0.0041
```

1. PREDICTION ON NEW DATA

```
print("\n")
print("EXAMPLE: PREDICTION ON NEW DATA")
print("\n \n")


sample_indices = np.random.choice(len(X_test), 5, replace=False)
X_sample = X_test.iloc[sample_indices]
y_sample = y_test[sample_indices]

print("\nSample Data for Prediction:")
for idx, sample_idx in enumerate(sample_indices, 1):
    true_label = label_encoder.inverse_transform([y_sample[idx-1]])[0]
    print(f"\nSample {idx}:")
    print(f"  Feature Values (first 6): {X_sample.iloc[idx-
1, :6].values}")
    print(f"  True Class: {true_label}")

    X_sample_scaled = scaler.transform(X_sample.iloc[[idx-1]])
    prediction = best_model.predict(X_sample_scaled)[0]
    probabilities = best_model.predict_proba(X_sample_scaled)[0]
    predicted_label = label_encoder.inverse_transform([prediction])[0]

    print(f"  Predicted Class: {predicted_label}")
    print(f"  Prediction Confidence:")
    for class_idx, class_name in enumerate(label_encoder.classes_):
        print(f"    - {class_name}: {probabilities[class_idx]:.4f}")

    match = "✓" if true_label == predicted_label else "✗"
    print(f"  Match: {match}")
```

```
EXAMPLE: PREDICTION ON NEW DATA



Sample Data for Prediction:

Sample 1:
  Feature Values (first 6): [-1.8096787  -1.4094648  -0.33883476
0.07528915  0.07330383  0.00297797]
  True Class: NORMAL
  Predicted Class: NORMAL
  Prediction Confidence:
    - AGGRESSIVE: 0.0000
    - NORMAL: 1.0000
    - SLOW: 0.0000
  Match: ✓

Sample 2:
  Feature Values (first 6): [ 0.5772822  -0.3661715  -1.2397652  -
0.19593503  0.4361578   0.13553573]
  True Class: AGGRESSIVE
  Predicted Class: AGGRESSIVE
  Prediction Confidence:
    - AGGRESSIVE: 0.9997
    - NORMAL: 0.0000
    - SLOW: 0.0003
  Match: ✓

Sample 3:
  Feature Values (first 6): [-0.41870224  0.05317104  2.2214575
0.18402316 -0.00916298  0.13492486]
  True Class: NORMAL
  Predicted Class: NORMAL
  Prediction Confidence:
    - AGGRESSIVE: 0.0000
    - NORMAL: 0.9999
    - SLOW: 0.0001
  Match: ✓

Sample 4:
  Feature Values (first 6): [-0.12730014 -0.08631774  0.3479681  -
0.01450805 -0.03420845  0.03474296]
  True Class: NORMAL
  Predicted Class: NORMAL
  Prediction Confidence:
    - AGGRESSIVE: 0.0000
    - NORMAL: 1.0000
    - SLOW: 0.0000
  Match: ✓
```

```
Sample 5:
  Feature Values (first 6): [ 0.54381895  0.31823707  0.27976608
0.05757405 -0.15454891 -0.06666067]
  True Class: AGGRESSIVE
  Predicted Class: AGGRESSIVE
  Prediction Confidence:
    - AGGRESSIVE: 1.0000
    - NORMAL: 0.0000
    - SLOW: 0.0000
  Match: ✓
```

Overfitting Check

```
from sklearn.model_selection import cross_val_score
from sklearn.metrics import accuracy_score, f1_score

print("="*60)
print("OVERFITTING CHECK")
print("="*60)

y_train_pred = best_model.predict(X_train_scaled)

train_acc = accuracy_score(y_train, y_train_pred)
train_f1 = f1_score(y_train, y_train_pred, average='weighted')

y_test_pred = best_model.predict(X_test_scaled)

test_acc = accuracy_score(y_test, y_test_pred)
test_f1 = f1_score(y_test, y_test_pred, average='weighted')

cv_scores = cross_val_score(best_model, X_train_scaled, y_train, cv=5,
scoring='f1_weighted')

print(f"\n TRAIN PERFORMANCE")
print(f"Accuracy : {train_acc:.4f}")
print(f"F1 Score : {train_f1:.4f}")

print(f"\n TEST PERFORMANCE")
print(f"Accuracy : {test_acc:.4f}")
print(f"F1 Score : {test_f1:.4f}")

print(f"\n CROSS-VALIDATION (5-Fold)")
print(f"Scores   : {cv_scores}")
print(f"Mean F1  : {cv_scores.mean():.4f}")

print("\n ANALYSIS")
print("-"*40)

train_test_gap = train_f1 - test_f1
```

```python
cv_test_gap = cv_scores.mean() - test_f1

print(f"Train - Test Gap : {train_test_gap:.4f}")
print(f"CV - Test Gap    : {cv_test_gap:.4f}")

if train_test_gap > 0.05:
    print(" Possible Overfitting (train >> test)")
elif test_f1 > train_f1:
    print(" Good Generalization (test >= train)")
else:
    print(" No significant overfitting detected")
```

```
============================================================
OVERFITTING CHECK
============================================================

 TRAIN PERFORMANCE
Accuracy : 1.0000
F1 Score : 1.0000

 TEST PERFORMANCE
Accuracy : 0.9959
F1 Score : 0.9959

 CROSS-VALIDATION (5-Fold)
Scores   : [0.9914304  0.99314674 0.99143219 0.9931407  0.99485033]
Mean F1  : 0.9928

 ANALYSIS
----------------------------------------
Train - Test Gap : 0.0041
CV - Test Gap    : -0.0031
 No significant overfitting detected
```